FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Fachhochschule Aachen

Campus Jülich

Fachbereich 9: Medizintechnik und Technomathematik
Studiengang: Angewandte Mathematik und Informatik

Flexible Verwaltung von Bauteildaten in CP-Pro CAD

Anforderungsanalyse mit anschließendem Entwurf eines Datenbanksystems

Bachelorarbeit

von

Matthias Rolfes

Erstprüfer:	Prof. Dr.-Ing. Andreas Terstegge
Zweitprüfer:	Wolfgang Malgadey, B. Sc.
Matrikelnummer:	3283725

Korschenbroich, den 2. Oktober 2023

Inhaltsverzeichnis

Abbildungsverzeichnis **iii**

1 Einleitung **1**
1.1 Aufbau der Arbeit 1

2 Einführung in die Software CP-Pro CAD **3**
2.1 Aufbau und Funktionsweise 5
2.2 Bauteilbibliothek 9
2.2.1 Eigenschaften von Bauteilen 9
2.2.2 Datenstruktur 13

3 Anforderungsanalyse **16**
3.1 Stakeholder 17
3.2 Anforderungen 19
3.3 Analyse 28
3.3.1 Strukturierung 28
3.3.2 Methodik 32
3.3.3 Bewertung 34
3.3.4 Priorisierung 38

4 Datenbanksystem als Lösungsentwurf **40**
4.1 Begründung der Auswahl 41
4.2 Umfang 42
4.3 Umsetzung 43
4.3.1 Datenbank 43

4.3.2 REST-API 45
4.3.3 Client 47
4.4 Diskussion und Ausblick 51

5 Fazit **53**

Abbildungsverzeichnis

2.1 Beispiel eines Fassadengerüsts in CP-Pro CAD 4
2.2 Generiertes Fassadengerüst mit Materialliste 6
2.3 Automatisches Einrasten einer Belagbohle 7
2.4 Treppenturm am Fassadengerüst 8

3.1 Anforderungen an die Bauteilbibliothek 29
3.2 Kosten-Nutzen-Matrix 39

4.1 Ausschnitt des Datenbankdiagramms 44
4.2 Dokumentation der REST-API in Swagger 46
4.3 Klassendiagramm des Clients 47
4.4 Übersicht auf die Bauteildaten im Client 48
4.5 Hinzufügen eines neuen Bauteils in der Bibliothekenverwaltung . . 49
4.6 Eintragen von Übersetzungen in der Ressourcenverwaltung 50

1 Einleitung

Während der stetigen Weiterentwicklung einer komplexen CAD-Software drehen sich viele Fragen um die zugehörige Bibliothek, welche die Daten zu allen Einzelbauteilen enthält. Mit der Zeit entstehen darin immer umfangreichere Strukturen mit zum Teil erheblichen Datenmengen. Eine Lösung zum möglichst flexiblen Umgang mit diesen Daten wird hierdurch zwingend erforderlich. Dabei treffen die verschiedensten Anforderungen aufeinander – sowohl von Anwendern als auch aus Unternehmenssicht.

Die Motivation dieser Arbeit liegt darin, die Idee für ein System zu finden, welches die wichtigsten Anforderungen abdeckt und zugleich genügend Erweiterungsmöglichkeiten bietet, um auch langfristig eine flexible Verwaltung von Bauteildaten zu gewähren. *Flexibilität* soll sich hierbei nicht nur auf die Daten beziehen, sondern auch auf das System an sich, um auf Änderungen der umgebenden Architektur jederzeit reagieren zu können.

1.1 Aufbau der Arbeit

Der Hauptteil der Arbeit gliedert sich in drei wesentliche Bereiche: Einführung in die Software, Anforderungsanalyse und Lösungsentwurf. Dabei nähert sich der Inhalt insgesamt von einer zunächst groben Skizzierung des Gesamtproblems schrittweise hin zur detaillierten Ausarbeitung einer Lösung als Antwort auf die zahlreichen Einzelfragen, die sich aus der Problemstellung ergeben.

Zu Beginn wird der Aufbau der Gerüstbauplanungssoftware *CP-Pro CAD* erklärt. Anhand mehrerer Screenshots, die verschiedene Anwendungssituationen darstellen, soll hierbei auch eine wesentliche Vorstellung von der Funktionsweise des Programms gewonnen werden. In einem gesonderten Abschnitt wird anschließend die Struktur der Bauteilbibliothek erörtert. Hier geht es vor allem darum, zu verstehen, welche Informationen über Bauteile gespeichert werden und wie die Daten miteinander verknüft sind.

Als Nächstes sollen die Anforderungen an ein flexibles Verwaltungssystem für Bauteildaten spezifiziert werden. Dazu werden die verschiedenen Stakeholder identifiziert und dann durch die Auswertung mehrerer Befragungen alle in diesem Zusammenhang bestehenden Anforderungen möglichst explizit dargestellt. Diese Anforderungen werden so strukturiert, dass daraus konkrete Änderungen und Erweiterungen abgeleitet werden können. Durch die jeweils individuell zu bewertenden Aspekte des Nutzens und des Aufwands sowie der Risiken ergibt sich nach einer sorgfältigen Analyse schließlich ein konkretes Bild davon, welche Anforderungen priorisiert werden müssen.

Abhängig von den aus der Analyse resultierenden Anforderungen an das Datenverwaltungssystem kann im letzten Schritt nun eine entsprechende Lösung entworfen werden. Mit Hilfe von UML-Modellierungstechniken wird das hierbei entstehende Produkt in seinen Grundzügen dokumentiert und die Anwendung des Systems veranschaulicht. Es folgt eine Diskussion über die Qualität der Lösung im Hinblick auf die tatsächliche Flexibilität der Datenverwaltung, bevor abschließend die wesentlichen Ergebnisse der Arbeit zusammengefasst werden.

2 Einführung in die Software CP-Pro CAD

CP-Pro CAD ist eine Software-Lösung zur Gerüstbauplanung, die im Jahre 2010 erstmals veröffentlicht wurde. Sie wird stetig weiterentwickelt und von der in Deutschland ansässigen CP-Pro Solutions GmbH herausgegeben. Bei der in C# programmierten WPF-Anwendung handelt es sich um ein klassisches CAD-Programm, wobei CAD für „*computer-aided design*“ (deutsch „rechnerunterstütztes Konstruieren“) steht und ein Sammelbegriff für diverse Aufgaben in der EDV ist, die mit Design, Konstruktion, Planung und Verbesserung neuer Produkte im Zusammenhang stehen. Konkret geht es hierbei vor allem um die geometrische Modellierung und Berechnung entsprechender Produkte, wodurch der Entwicklungsprozess von Produktkonstruktionen erheblich beschleunigt wird. Oftmals werden dabei viele einzelne Materialien als dreidimensionale Körper zusammengesetzt und erzeugen letztendlich ein komplexes, detailliertes Gebilde. Durch die Erfassung von Daten über die Eigenschaften der Einzelteile und deren computergestützte Verarbeitung können zusätzliche Informationen über das Endprodukt und seine physikalischen Merkmale gewonnen werden.

Diese Aufgaben erfüllt CP-Pro CAD spezifisch für die Gerüstbaubranche. Der Benutzer kann beliebige Fassaden- und Raumgerüste konstruieren oder durch das Programm mit Hilfe von importierten Grundrisszeichnungen von geplanten Gebäuden konstruieren lassen (siehe Abbildung 2.1). Dabei steht ihm eine Auswahl aus zahlreichen Bauteilen von mehreren Herstellern zur Verfügung. Mit Hilfe von Assistenzfunktionen können Gerüste auf verschiedene Arten ausgebaut werden,

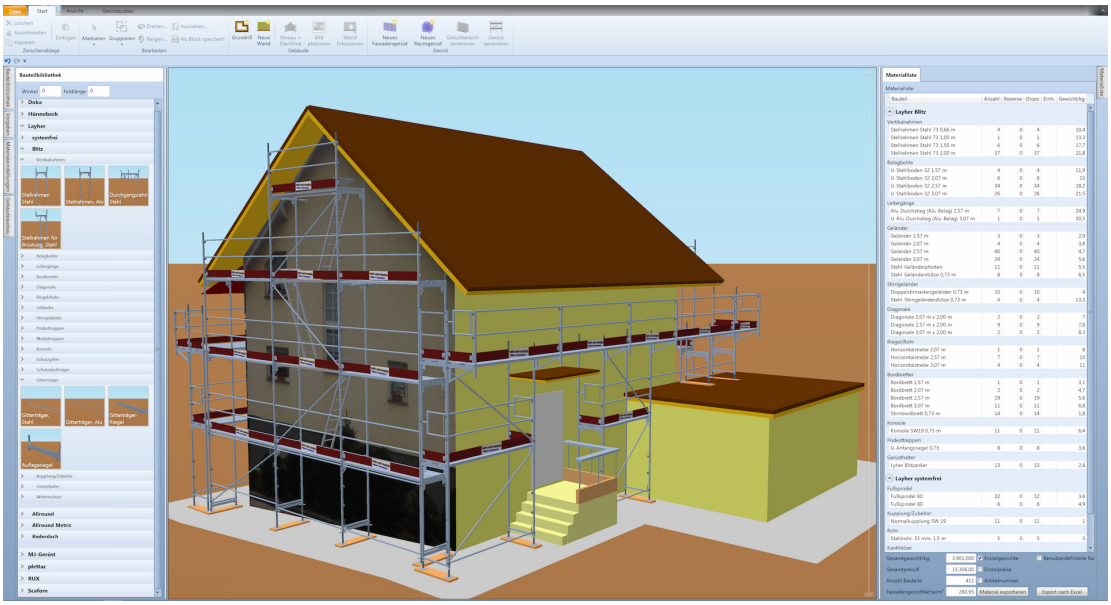


Abbildung 2.1: Beispiel eines Fassadengerüsts in CP-Pro CAD

wobei die einzelnen Bauteile spezifische Rastpunkte besitzen, sodass sie sich passend zusammensetzen lassen. Außerdem übernimmt das Programm die Ermittlung des Materialbedarfs und berechnet dabei durch die mitgelieferten Daten der Bauteile auch wichtige Kennzahlen wie z. B. die Gesamtkosten.

2.1 Aufbau und Funktionsweise

Die Software besitzt einen für CAD typischen Aufbau. Hierbei lassen sich die wesentlichen Hauptkomponenten, aus denen sie zusammengesetzt ist, klar benennen. Deren Funktionsumfang wird im Folgenden jeweils grob erklärt.

Benutzeroberfläche

Die Benutzeroberfläche dient der Interaktion mit dem Benutzer. Sie umfasst eine Szene zur Modelldarstellung im dreidimensionalen Raum, die beliebig bearbeitet werden kann, sowie mehrere Menüs mit Funktionen, um das Programm zu steuern. Darüber kann der Benutzer z. B. Einstellungen am Programm vornehmen, die Elemente des Modells auf verschiedene Weisen bearbeiten oder die Ansicht konfigurieren. Außerdem werden über die Benutzeroberfläche Daten zum Modell und andere Informationen in einer Materialliste zur Verfügung gestellt (siehe Abbildung 2.2).

Bauteilbibliothek

Die Bauteilbibliothek ist eine strukturierte Sammlung vordefinierter Gerüstbauteile von verschiedenen Herstellern, die verwendet werden können, um ein komplexes Gerüst zusammenzubauen. Sie enthält sämtliche Eigenschaften der Bauteile, die für die Software relevant sein könnten. Dazu gehören neben den grundlegenden Klassifizierungsdaten und visuellen Merkmalen auch diverse Informationen zur genauen Verwendung der Bauteile in einem Gerüst sowie deren Verbindungspunkte, damit sie sich durch das Programm intelligent zusammensetzen lassen. Die Beschaffenheit der Bauteilbibliothek ist für das Thema der Bauteildatenverwaltung von besonders essenzieller Bedeutung, weshalb sie in Kapitel 2.2 noch einmal ausführlich erörtert wird.

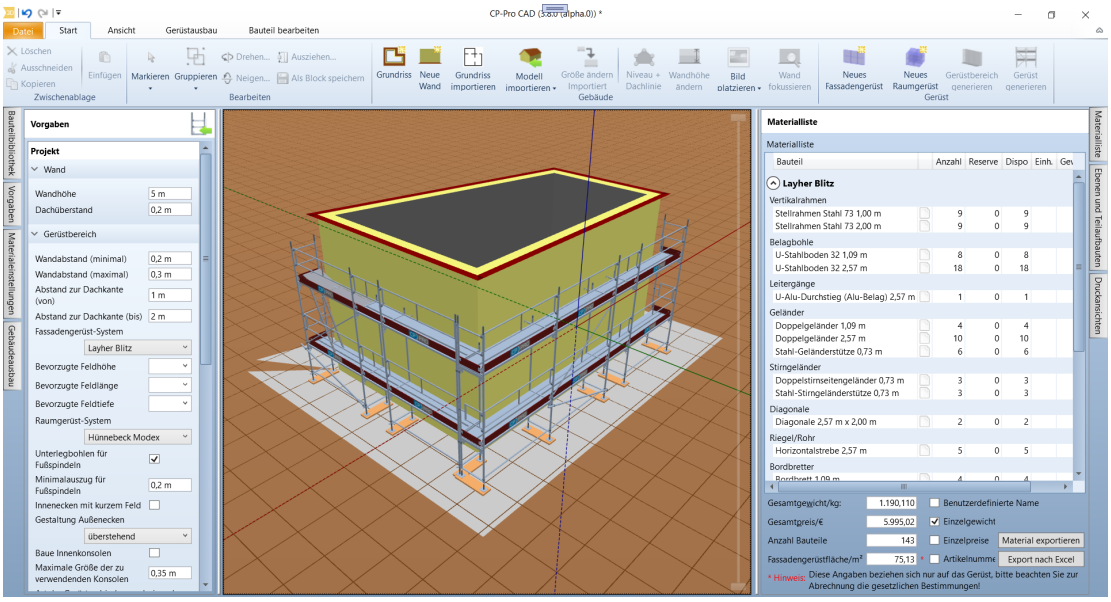


Abbildung 2.2: Generiertes Fassadengerüst mit Materialliste

Geometrie-Engine

Die Geometrie-Engine ist für die Berechnung und Speicherung der geometrischen Informationen des Modells verantwortlich. Sie enthält zahlreiche Algorithmen, welche die korrekte Durchführung von Anpassungen am Modell im dreidimensionalen Raum ermöglichen. Dazu gehören sowohl Basisoperationen wie Verschiebungen und Drehungen als auch komplexere Aufgaben wie z. B. die automatische Erkennung von Flächen oder Ausrichtungen. Damit wird die Basis für sämtliche Funktionen zur Gerüstkonstruktion gelegt.

Konstruktionswerkzeuge

Als Konstruktionswerkzeuge werden grundsätzlich alle Funktionen bezeichnet, die dazu dienen, einzelne Elemente eines Modells zu erstellen und zu verändern. In CP-Pro CAD fällt darunter vor allem der Umgang mit einzelnen Bauteilen. Diese können zur Szene hinzugefügt, darin beliebig bewegt oder wieder entfernt werden (siehe Abbildung 2.3). In einem Bauteileditor besteht außerdem die Möglichkeit, die

Eigenschaften der Bauteile zu bearbeiten. Der Benutzer kann auch Hilfselemente wie z. B. Wände zeichnen und dadurch konstruieren, um auf Grundlage dessen ein Gerüst generieren zu lassen, oder beliebige Beschriftungen in das Modell einfügen.

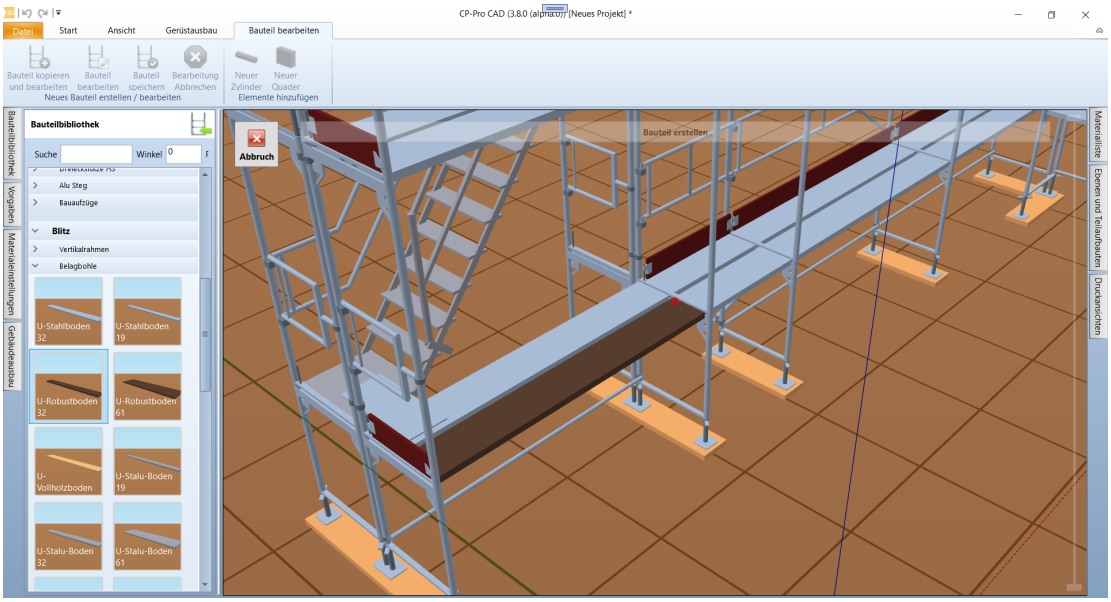


Abbildung 2.3: Automatisches Einrasten einer Belagbohle

Assistenzfunktionen

Die Assistenzfunktionen bieten dem Benutzer die Option, umfangreiche Prozesse bei der Erstellung und dem Ausbau eines Gerüsts automatisiert durch das Programm vornehmen zu lassen. Die Software enthält bereits zahlreiche Vorgaben, die für den Gerüstbau gelten, und lässt die Konfiguration weiterer individueller Vorgaben durch den Benutzer zu. Mit Hilfe dieser Vorgaben und den in der Bauteilbibliothek enthaltenen Informationen zur Verwendung von Bauteilen und Gerüstsystemen kann das Programm die Bauteile ordnungsgemäß zusammensetzen und die entsprechenden Modellierungen vollziehen (siehe Abbildung 2.4).

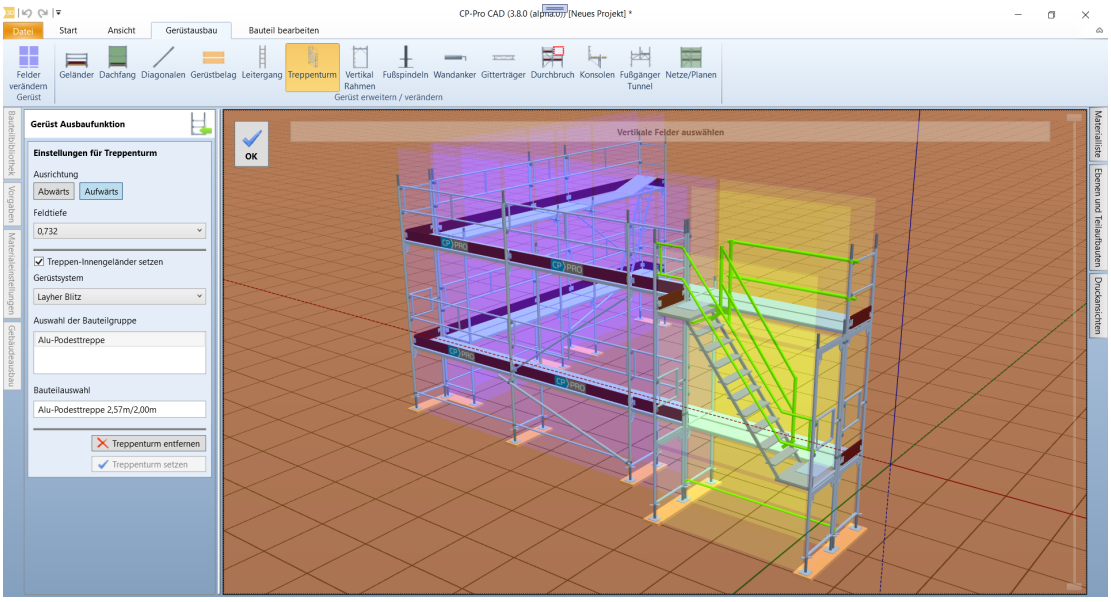


Abbildung 2.4: Treppenturm am Fassadengerüst

Ein- und Ausgabe

Zu der Ein- und Ausgabe gehören verschiedene Möglichkeiten, um Modelldaten in das Programm zu importieren oder aus diesem zu exportieren. Der Benutzer kann Grundrisszeichnungen oder ganze Gebäudemodelle in verschiedenen Formaten in die Software laden, um auf deren Grundlage ein Gerüst zu erstellen. Fertige Gerüstkonstruktionen können wiederum in einem einheitlichen Format ausgegeben werden. Das Programm kann verschiedene Ausdrucke erzeugen, sowohl von 2D- und 3D-Ansichten als auch von Materiallisten des Gerüsts. Außerdem besteht die Möglichkeit, einzelne Bauteile oder Bauteilgruppen abzuspeichern und wiederzuverwenden.

2.2 Bauteilbibliothek

Sämtliche Daten zu vordefinierten Bauteilen in CP-Pro CAD sind in einer Bauteilbibliothek gespeichert. Dabei handelt es sich um einen strukturierten Datensatz, der nicht nur Informationen zu den Bauteilen an sich beinhaltet, sondern auch zu deren zum Teil mehrfach verwendeten Visualisierungen und Verbindungspunkten sowie zu verschiedenen Gruppierungen wie z. B. Bauteilklassen und Gerüstsystemen.

Welche Daten in der Bibliothek genau enthalten sind, wird grundsätzlich durch das Produktmanagement festgelegt und durch die Entwickler der Software realisiert. Die gesamte Bauteilbibliothek wird bei der Installation der Software mitgeliefert und liegt dann in Form mehrerer .xml-Dateien lokal auf dem jeweiligen Rechner vor. Beim Start der Software werden sämtliche Daten aus diesen Dateien in ein Objekt vom Typ *PartLibrary* geladen, mit welchem das Programm arbeiten kann. Für den Anwender besteht nachträglich die Möglichkeit, die Daten zu erweitern, indem er im Bauteileditor des Programms eigene Bauteile erzeugt und diese entsprechend abspeichert.

2.2.1 Eigenschaften von Bauteilen

Bauteile in CP-Pro CAD besitzen zahlreiche Eigenschaften, die sich in drei Bereiche grob unterteilen lassen: Kennzeichnungseigenschaften, Verwendungseigenschaften und Visualisierungseigenschaften. Um diese Eigenschaften vollständig abzubilden, existiert in der Bauteilbibliothek eine Reihe von Objektklassen, die wiederum eigene Eigenschaften besitzen.

Die Objektklassen lauten wie folgt:

- ShapeColor
- UnitMesh
- MeshVertex
- MeshTriangle
- VisualizationShape
- Visualization
- Maker
- ScaffoldSystem
- PartClass
- PartGroup
- FreePart
- Part
- UuidSubstitution
- ConnectionPoint
- ConnectionFlex
- ConnectionOffset

Jedes Objekt einer solchen Klasse spielt für die Definition einzelner Bauteile eine wesentliche Rolle. Die Bauteileigenschaften sind also nicht nur im *Part* selbst definiert, sondern es bestehen direkte und indirekte Verknüpfungen vom Part zu allen anderen Klassen. Diese Verknüpfungen werden in Kapitel 2.2.2 im Detail skizziert. Im Folgenden werden zunächst die wichtigsten Eigenschaften im Überblick beschrieben, wobei für alle Bereiche jeweils das Part-Objekt als Startpunkt betrachtet wird, um davon ausgehend den Bezug zu anderen für diese Eigenschaften relevanten Objekten darzustellen.

Kennzeichnungseigenschaften

Die Kennzeichnungseigenschaften setzen sich aus einer eindeutigen Identifizierung, einer Reihe von Metadaten und einer klaren Zuordnung zusammen. Die Identifizierung eines Bauteils erfolgt zum einen durch seine ID und zum anderen durch seine UUID (*Universally Unique Identifier*). Während die ID nur innerhalb einer .xml-Datei eindeutig sein muss, die in der Regel pro Hersteller vorliegt, wird die UUID für die Referenzierung im laufenden Programm verwendet und muss daher im gesamten System eindeutig sein. Musste eine UUID nachträglich verändert werden, wurde dafür ein entsprechendes Objekt angelegt (*UuidSubstitution*), welches sowohl die alte als auch die neue UUID enthält, damit bei bereits bestehden Projekten mit dem betroffenen Bauteil keine Fehler auftreten.

Jedes Bauteil (*Part*) ist einem freien Bauteil (*FreePart*) zugeordnet. Dabei handelt es sich um ein noch nicht im Raum angeordnetes Bauteil, welches also keine Angaben zur Ausdehnung und Orientierung besitzt. Diese Klasse stellt den Großteil der Metadaten zur Verfügung. Dazu gehören der Name, die Artikelnummer, das Gewicht und der Preis des Bauteils. Lediglich ein Namenssuffix, welches z. B. die Länge des Bauteils enthalten könnte, ist im Part selbst enthalten.

Die Zuordnung des Bauteils ist durch die Beziehungen zu den jeweils übergeordneten Objekten der Bauteilbibliothek gegeben. Jedes freie Bauteil gehört einer Bauteilgruppe (*PartGroup*) an und diese wiederum einer Bauteilklasse (*PartClass*). Bauteile innerhalb einer Bauteilgruppe stellen verschiedene Versionen eines Bauteils dar und unterscheiden sich in der Regel vor allem durch ihre Größe, während Bauteilklassen jeweils Bauteile mit ähnlichen Eigenschaften und häufig gleichem Nutzen zusammenfassen (z. B. Fußspindel, Rohr, Belagbohle). Bauteilklassen gehören immer zu einem Gerüstsystem (*ScaffoldSystem*) und jedes Gerüstsystem besitzt einen Hersteller (*Maker*).

Verwendungseigenschaften

Durch eine Reihe von Verwendungseigenschaften wird festgelegt, auf welche Weise das jeweilige Bauteil im Programm benutzt werden kann. Dies ist vor allem für die Assistenzfunktionen von essenzieller Bedeutung, da sie erkennen müssen, welches Bauteil eine bestimmte Rolle erfüllen kann und an welcher Position dieses eingefügt werden soll. Entscheidend für die Rolle ist vor allem das Attribut *Usage*, welches sowohl für eine Bauteilgruppe als auch für eine gesamte Bauteilklasse angegeben werden kann. Außerdem ist die eindeutige Zuordnung zum Gerüstsystem relevant für die Verwendung des Bauteils.

Jedem Bauteil sind beliebig viele Verbindungszeiger (*ConnectionOffset*) zugeordnet. Diese geben jeweils eine relative Position zum Bauteil an und erzeugen an dieser Position einen Verbindungspunkt (*ConnectionPoint*). Dort ist dann die Einrastung an einem anderen Bauteil möglich, welches einen dazu passenden Verbindungspunkt besitzt. Ob zwei Verbindungspunkte zueinanderpassen, hängt von den darin enthaltenen Eigenschaften fest. Dazu gehören der Winkel, die Neigung und der durch eine Ganzzahl angegebene Typ (Beispiel: Typ 24 passt zu Typ -24). Zusätzlich kann dem Verbindungspunkt eine Verbindungsspanne (*ConnectionFlex*) zugeordnet sein, die den Punkt in einer Dimension ausdehnt.

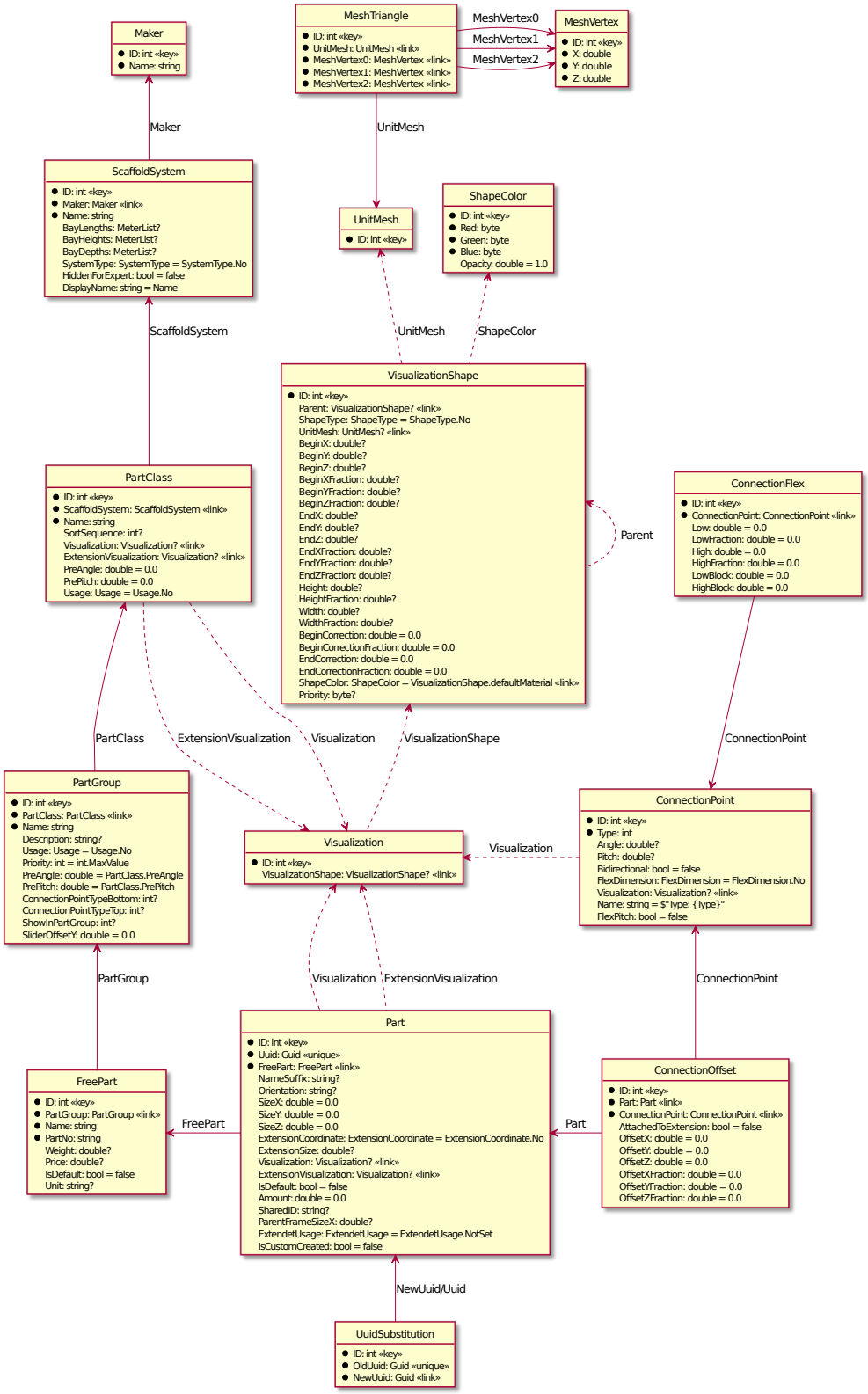
Visualisierungseigenschaften

Zu den Visualisierungseigenschaften gehören alle Eigenschaften, die die grafische Darstellung des Bauteils im Programm bestimmen. Das Bauteil selbst enthält hierzu Attribute zur Ausdehnung in den drei Dimensionen. Für eine Bauteilgruppe und für eine Bauteilklasse können Winkel und Neigung vordefiniert werden. Bauteilklassen, einzelne Bauteile und auch Verbindungspunkte können eine Visualisierung (*Visualization*) besitzen und jede Visualisierung greift auf eine Visualisierungsgestaltung (*VisualizationShape*) zu. Dabei werden einem Hauptelement in der Regel mehrere Kindelemente zugeordnet, die gemeinsam die Visualisierung des gesamten Bauteils oder des Verbindungspunktes ergeben.

Objekte zur Visualisierungsgestaltung besitzen grundsätzlich eine Farbe (*ShapeColor*) im RGB-Format und einen bestimmten *ShapeType*. Dies ist entweder ein Quader, ein Zylinder oder ein Dreiecksnetz. In den ersten beiden Fällen werden die Abmessungen über absolute und relative Angaben zu Start- und Endpunkten in allen drei Dimensionen festgelegt. Im Falle eines Dreiecksnetzes wird ein solches Mesh (*UnitMesh*) angegeben. Dem Mesh werden beliebig viele Dreiecke (*MeshTriangle*) zugeordnet, welche dann das Gesamtnetz bilden. Jedes dieser Dreiecke besteht aus drei Eckpunkten (*MeshVertex*) mit X-, Y- und Z-Koordinaten.

2.2.2 Datenstruktur

Die Bauteilbibliothek liegt im Dateisystem in Form mehrerer .xml-Dateien vor (grundsätzlich eine Datei pro Hersteller). Nach dem Programmstart werden daraus entsprechende Objekte im Speicher erzeugt. Der XML-Datensatz ist so definiert, dass alle in Kapitel 2.2.1 aufgelisteten Objekte der Bibliothek als Elemente durch entsprechende Tags nacheinander, also ohne Verschachtelung deklariert werden. Die Eigenschaften der einzelnen Objekte sind darin jeweils ebenfalls als Elemente mit Anfangstags und Endtags realisiert (z. B. *<ID>24</ID>*). Konzeptuell lässt sich die Bauteilbibliothek somit als Abbild einer Datenbank betrachten, wobei jedes Grundelement eine Zeile einer entsprechenden Tabelle widerspiegelt und jede Eigenschaft in einer Spalte dieser Tabelle liegt. Über die IDs als Schlüsselattribute besitzen die Tabellen relationale Beziehungen zueinander. Ausgehend von diesem Konzept ergibt sich für die Bauteilbibliothek folgende Datenstruktur:


Maker
ID: int «key»
Name: string
MeshTriangle
ID: int «key»
UnitMesh: UnitMesh «link»
MeshVertex0: MeshVertex «link»
MeshVertex1: MeshVertex «link»
MeshVertex2: MeshVertex «link»
MeshVertex0
MeshVertex1
MeshVertex2
MeshVertex
ID: int «key»
X: double
Y: double
Z: double
Maker
UnitMesh
ScaffoldSystem
ID: int «key»
Maker: Maker «link»
Name: string
BayLengths: MeterList?
BayHeights: MeterList?
BayDepths: MeterList?
SystemType: SystemType = SystemType.No
HiddenForExpert: bool = false
DisplayName: string = Name
UnitMesh
ID: int «key»
ShapeColor
ID: int «key»
Red: byte
Green: byte
Blue: byte
Opacity: double = 1.0
ScaffoldSystem
UnitMesh
ShapeColor
VisualizationShape
ID: int «key»
Parent: VisualizationShape? «link»
ShapeType: ShapeType = ShapeType.No
UnitMesh: UnitMesh? «link»
BeginX: double?
BeginY: double?
BeginZ: double?
BeginXFraction: double?
BeginYFraction: double?
BeginZFraction: double?
EndX: double?
EndY: double?
EndZ: double?
EndXFraction: double?
EndYFraction: double?
EndZFraction: double?
Height: double?
HeightFraction: double?
Width: double?
WidthFraction: double?
BeginCorrection: double = 0.0
BeginCorrectionFraction: double = 0.0
EndCorrection: double = 0.0
EndCorrectionFraction: double = 0.0
ShapeColor: ShapeColor = VisualizationShape.defaultMaterial «link»
Priority: byte?
Parent
PartClass
ID: int «key»
ScaffoldSystem: ScaffoldSystem «link»
Name: string
SortSequence: int?
Visualization: Visualization? «link»
ExtensionVisualization: Visualization? «link»
PreAngle: double = 0.0
PrePitch: double = 0.0
Usage: Usage = Usage.No
ConnectionFlex
ID: int «key»
ConnectionPoint: ConnectionPoint «link»
Low: double = 0.0
LowFraction: double = 0.0
High: double = 0.0
HighFraction: double = 0.0
LowBlock: double = 0.0
HighBlock: double = 0.0
PartClass
ExtensionVisualization
Visualization
VisualizationShape
ConnectionPoint
PartGroup
ID: int «key»
PartClass: PartClass «link»
Name: string
Description: string?
Usage: Usage = Usage.No
Priority: int = int.MaxValue
PreAngle: double = PartClass.PreAngle
PrePitch: double = PartClass.PrePitch
ConnectionPointTypeBottom: int?
ConnectionPointTypeTop: int?
ShowInPartGroup: int?
SliderOffsetY: double = 0.0
Visualization
ID: int «key»
VisualizationShape: VisualizationShape? «link»
Visualization
ConnectionPoint
ID: int «key»
Type: int
Angle: double?
Pitch: double?
Bidirectional: bool = false
FlexDimension: FlexDimension = FlexDimension.No
Visualization: Visualization? «link»
Name: string = $"Type: {Type}"
FlexPitch: bool = false
PartGroup
Visualization
ExtensionVisualization
ConnectionPoint
Part
ID: int «key»
Uuid: Guid «unique»
FreePart: FreePart «link»
NameSuffix: string?
Orientation: string?
SizeX: double = 0.0
SizeY: double = 0.0
SizeZ: double = 0.0
ExtensionCoordinate: ExtensionCoordinate = ExtensionCoordinate.No
ExtensionSize: double?
Visualization: Visualization? «link»
ExtensionVisualization: Visualization? «link»
IsDefault: bool = false
Amount: double = 0.0
SharedID: string?
ParentFrameSizeX: double?
ExtendetUsage: ExtendetUsage = ExtendetUsage.NotSet
IsCustomCreated: bool = false
FreePart
ID: int «key»
PartGroup: PartGroup «link»
Name: string
PartNo: string
Weight: double?
Price: double?
IsDefault: bool = false
Unit: string?
FreePart
Part
ConnectionOffset
ID: int «key»
Part: Part «link»
ConnectionPoint: ConnectionPoint «link»
AttachedToExtension: bool = false
OffsetX: double = 0.0
OffsetY: double = 0.0
OffsetZ: double = 0.0
OffsetXFraction: double = 0.0
OffsetYFraction: double = 0.0
OffsetZFraction: double = 0.0
NewUuid/Uuid
UuidSubstitution
ID: int «key»
OldUuid: Guid «unique»
NewUuid: Guid «link»

Die nachgestellte Angabe *«key»* bedeutet, dass es sich bei der Spalte um einen Primary Key handelt (trifft ausschließlich auf IDs zu). Werte einer Spalte mit der Angabe *«unique»* müssen in der gesamten Bauteilbibliothek eindeutig sein (UUIDs). Ein ausgefüllter Kreis • vor einem Spaltennamen bedeutet, dass dieser Wert angegeben werden muss. Anderenfalls ist die Angabe optional. Bei fehlender Angabe wird der Wert auf *null* gesetzt oder – falls vorhanden – auf den Defaultwert. Ein durchgezogener Pfeil von einer Tabelle zu einer anderen Tabelle gibt einen obligatorischen Link an und ein gestrichelter Pfeil einen optionalen Link. Dabei wird jeweils die Tabelle, von der der Pfeil ausgeht, als Quelltabelle des Links bezeichnet; die Tabelle, zu der der Pfeil hinweist, als Zieltabelle des Links. Für beide Arten von Links sind der Name der Spalte in der Quelltabelle und der Name der Spalte in der Zieltabelle durch einen Schrägstrich getrennt am Pfeil notiert. Dabei entfällt die Angabe der Spalte in der Zieltabelle, wenn es sich um den Primary Key der Zieltabelle handelt. Die Spaltendefinition in der Quelltabelle wird von der Angabe *«link»* gefolgt. Die durch Pfeile symbolisierten Links sind bei Betrachtung als Datenbank 1:n-Relationen (oder 0..1:n-Relationen bei optionalen Links), bei denen die Pfeilspitze mit einer *1* (oder *0..1* bei optionalen Links) und der Ausgangspunkt des Pfeils mit einem n versehen werden könnte.

Um als Entwickler Bauteildaten zu editieren oder hinzuzufügen, ist es bisher notwendig, manuelle Veränderungen an mehreren Stellen im XML-Datensatz vorzunehmen. Jedes Element, welches für das Bauteil benötigt wird, muss einzeln in die Struktur eingefügt werden. Dabei geht es in der Regel pro Bauteil mindestens um eine Visualization, mehrere VisualizationShapes, ein FreePart, ein Part und mehrere ConnectionOffsets. Häufig wird für einen Teil dieser Elemente ein bereits vorhandes Objekt per Copy & Paste als Vorlage benutzt und daran anschließend geringfügige Änderungen vorgenommen. Dabei muss für jedes neu erstellte Element jeweils eine eindeutige ID vergeben werden und die verwendeten Elemente müssen per Hand über ihre IDs referenziert werden. Alle Updates, welche die Bauteildaten betreffen, werden im Zuge von generellen Softwareupdates ausgeliefert, die in Abständen von zumeist mehreren Monaten dem Anwender zur Verfügung gestellt werden.

3 Anforderungsanalyse

Für den Entwurf eines flexiblen Verwaltungssystems für Bauteildaten ist es notwendig, vorab die konkreten Anforderungen an ein solches System zu spezifizieren. Hierzu müssen zunächst die wesentlichen Stakeholderklassen identifiziert werden. Die Anforderungen der verschiedenen Stakeholder werden dann mit Hilfe entsprechender Befragungen ermittelt und möglichst stark verfeinert. Anschließend werden die Anforderungen strukturiert und daraus abgeleitete Features im Hinblick auf entscheidende Bewertungskriterien analysiert. Dabei findet – ähnliche wie in einer klassischen Nutzwertanalyse – eine festgelegte Gewichtung der einzelnen Kriterien statt, sodass im Resultat für jedes Feature eine aussagekräftige Gesamtbewertung entsteht. Aus dieser Bewertung lässt sich die jeweilige Priorisierung ableiten, sodass deutlich wird, welche Anforderungen das System zwingend erfüllen muss und worin eine entsprechende Lösung bestehen könnte.

3.1 Stakeholder

Die Stakeholder, also alle Personen mit berechtigtem Interesse an einem flexiblen Verwaltungssystem für Bauteildaten, entsprechen weitestgehend den Stakeholdern der Software im Allgemeinen. Hierfür kommen also sämtliche Personen in Frage, die mit der Software CP-Pro CAD und speziell der Bauteilbibliothek direkt oder indirekt in Berührung kommen. Dies sind zum einen alle Mitarbeiter der vertreibenden CP-Pro Solutions GmbH sowie alle an der Entwicklung beteiligten Personen (Mitarbeiter der CP-Pro Software GmbH) und zum anderen alle Anwender der Software. Außerdem könnten Bauteilhersteller, die ihre Bauteile für die Software zur Verfügung stellen, bestimmte Interessen daran besitzen.

Bei genauerer Betrachtung lassen sich die verschiedenen Personengruppen eindeutig klassifizieren und zum Teil reduzieren. Die Interessen des Vertriebs werden von der Geschäftsführung klar vorgegeben. Die Entwickler können als eigene Stakeholderklasse aufgefasst werden, da sie durch ihre ständige Arbeit an der Software spezifische Anforderungen an die Architektur stellen, welche ihnen die Arbeit erleichtern können. Die Entscheidungsfindung darüber liegt zu einem erheblichen Anteil beim Produktmanagement. Hierbei fließen auch eigene Vorstellungen mit ein, die der Product Owner von der Architektur und der langfristigen Weiterentwicklung der Software besitzt, sowie diverse Anforderungen der Anwender. Die Interessen der Bauteilhersteller werden im Wesentlichen auch von Geschäftsführung und Produktmanagement vertreten, weshalb sie hier nicht weiter aufgeführt werden. Somit ergeben sich folgende Stakeholderklassen:

Stakeholder: Geschäftsführung

Ziel: Gewinnoptimierung durch attraktive Produktgestaltung und Einsparung von Ressourcen

Stakeholder: Produktmanagement

Ziel: Optimierung der Softwarearchitektur zur Schaffung einer möglichst hohen Anpassungsfähigkeit

Stakeholder: Entwickler

Ziel: Vereinfachung der Arbeit beim Erstellen und Ändern von Bauteilen

Stakeholder: Anwender

Ziel: Erhöhung der Benutzerfreundlichkeit durch mehr Optionen zur Konfiguration des Programms

3.2 Anforderungen

Die verschiedenen Stakeholder stellen jeweils eigene Anforderungen an das Bauteildatensystem. Für die Ermittlung aller Anforderungen wurden Befragungen des Geschäftsführers sowie des Product Owners durchgeführt. Der Product Owner berichtete hierbei nicht nur über die Anforderungen des Produktmanagements, sondern auch stellvertretend über die der Entwickler und der Anwender. Die Anforderungen wurden in den Befragungen zum Teil nur grob ausgeführt und werden in diesem Abschnitt verfeinert, um darüber anschließend möglichst präzise Aussagen treffen zu können.

Berücksichtigt in der folgenden Zusammenstellung werden all jene Anforderungen, die sich spezifisch an die Implementierung von neuen Features im Zusammenhang mit der Bauteilbibliothek richten. Im Anschluss daran werden weitere nicht-funktionale Anforderungen genannt, welche die allgemeine Produktentwicklung betreffen und nicht explizit als Anforderungen an ein Verwaltungssystem für Bauteildaten formuliert wurden, möglicherweise aber dennoch eine grundlegende Relevanz für die Auswahl der Implementierung besitzen.

Die einzelnen Anforderungen werden mit Zahlen nummeriert, wobei Teilanforderungen durch eine entsprechende Unternummerierung angegeben werden (z. B. Anforderung 2.1 als Teilanforderung der Anforderung 2).

Anforderung	**1) Vereinfachung bei der Implementierung von Bauteilen**
Beschreibung	Neue Bauteile sollen leichter hinzugefügt werden können.
Stakeholder	Geschäftsführung, Produktmanagement, Entwickler
Spezifizierung	Derzeit sind die Strukturen so komplex, dass Bauteile oder Teile von Bauteilen schlecht wiederverwendet werden können, um neue Bauteile zu erstellen oder davon abzuleiten. Zur Erstellung der Bauteile ist nicht nur das Wissen über die Struktur der Bauteilbibliothek erforderlich, sondern auch die Auswirkungen sehr vieler einzelner Parameter und wie diese in den Assistenzfunktionen wirken. Diese Auswirkungen sind während der Implementierung der Bauteile nicht direkt sichtbar. Fehler haben oft massive Probleme zur Folge, die erst viel später festgestellt werden.
Teilanforderungen	1.1) Vereinfachung der Datenstruktur 1.2) Hinzufügen von Informationen zu den Auswirkungen einzelner Parameter 1.3) Automatische Validierung der implementierten Bauteile

Anforderung	**1.1) Vereinfachung der Datenstruktur**
Beschreibung	Die Struktur der Bauteildaten soll vereinfacht werden.
Stakeholder	Produktmanagement, Entwickler
Spezifizierung	Momentan ist die Datenstruktur der Bauteilbibliothek nicht optimal. Das XML-Format wird nicht effizient genutzt, da die Elemente nicht sinnvoll verschachtelt werden und keine Attribute enthalten. Dadurch besteht eine unnötig hohe Komplexität vor allem in Bezug auf IDs, welche sich auch auf einen hohen Aufwand für die Implementierung von neuen Bauteilen auswirkt. Die Struktur soll derart vereinfacht werden, dass sich möglichst viele Informationen zu einem Bauteil am gleichen Ort befinden. Das Programm muss auf die neue Struktur abgestimmt werden, gleichzeitig aber auch die alte Struktur berücksichtigen, die bei benutzerdefinierten Bauteilen des Anwenders weiterhin vorliegt.

Anforderung	**1.2) Hinzufügen von Informationen zu den Auswirkungen einzelner Parameter**
Beschreibung	Für das Erstellen von Bauteilen sollen Hinweise zu den Auswirkungen einzelner Parameter gegeben werden.
Stakeholder	Entwickler
Spezifizierung	Die Implementierung neuer Bauteile führt derzeit häufig zu schwerwiegenden Fehlern, welche erst deutlich später bemerkt werden, weil die Angabe bestimmter Parameter zum Teil spezifisches Fachwissen erfordert und die Auswirkungen nicht direkt sichtbar sind. Vor allem die einzelnen Reaktionen der Assistenzfunktionen sind für den Entwickler oft unklar. Daher soll die korrekte Auswahl der Parameter erleichtert werden, indem diese z. B. automatisch eingeschränkt wird oder nützliche Anmerkungen angezeigt werden.

Anforderung	**1.3) Automatische Validierung der implementierten Bauteile**
Beschreibung	Bauteildaten sollten im Hinblick auf das Format automatisch validiert werden.
Stakeholder	Entwickler
Spezifizierung	Auf eine Nichteinhaltung des korrekten Formats (momentan XML) wird der Entwickler bisher in keiner Form hingewiesen. Erst nach dem Start des Programms wird teilweise durch eine fehlende Anzeige der Bauteilbibliothek ersichtlich, dass die Struktur fehlerhaft ist. Um mögliche Fehler frühzeitig zu vermeiden, sollte eine automatisierte Prüfung des Formats stattfinden.

Anforderung	**2) Vereinfachung bei der Aktualisierung von Bauteildaten**
Beschreibung	Die Aktualisierung von Preisen und ggf. auch anderen Bauteildaten soll möglichst automatisiert werden.
Stakeholder	Geschäftsführung, Produktmanagement
Spezifizierung	Aktuell wird die Aktualisierung der Preise von der Qualitätssicherung durchgeführt, wenn Preislisten verfügbar sind. Diese können in verschiedenen Formaten vorliegen. Optimal wäre die automatische Verwendung der Preise aus der Herstellerdatenbank. Es kommt vor, dass mit neuen Preislisten auch Gewichte und Artikelnummern von Bauteilen angepasst werden. Dabei handelt es sich um neue Versionen von Bauteilen.
Teilanforderungen	2.1) Verknüpfung verschiedener Versionen von Bauteilen

Anforderung	**2.1) Verknüpfung verschiedener Versionen von Bauteilen**
Beschreibung	Neue Versionen eines Bauteils sollen jeweils mit ihrer alten Version verknüpft werden.
Stakeholder	Produktmanagement
Spezifizierung	Wenn neue Versionen von Bauteilen erscheinen, erhalten diese eine neue Artikelnummer und häufig auch sonstige veränderte Metadaten. Die alte Version soll in diesem Fall nicht ersetzt werden, da ein Anwender nicht zwingend die neue Version benutzen möchte. Verschiedene Versionen eines Bauteils sollen in den Daten miteinander verknüpft werden, um einen entsprechenden Umgang zu ermöglichen.

Anforderung	**3) Bereitstellung von Bauteildaten unabhängig von Softwareupdates**
Beschreibung	Bauteildaten sollen unabhängig von vollständigen Softwareupdates online zum Download verfügbar sein.
Stakeholder	Geschäftsführung, Anwender
Spezifizierung	Bisher wird die Aktualisierung der Bauteilbibliothek ausschließlich bei einer Aktualisierung der Software vorgenommen. Die entsprechenden Releasezyklen dauern oft mehrere Monate. Anwender sollen unabhängig davon neue und aktualisierte Bauteile herunterladen können. Es könnten auch nur die Bauteile heruntergeladen werden, die benötigt werden, oder kostenpflichtige Angebote bereitgestellt werden.
Teilanforderungen	3.1) Auswahl von benötigten Bauteilen zum Download 3.2) Konfiguration von Berechtigungen zum Zugriff auf Bauteildaten

Anforderung	**3.1) Auswahl von benötigten Bauteilen zum Download**
Beschreibung	Anwender sollen die Bibliothek auf ihre individuell benötigten Bauteile reduzieren können.
Stakeholder	Anwender
Spezifizierung	Viele Anwender benötigen nicht die gesamte Bauteilbibliothek, sondern benutzen nur bestimmte Bauteile von einzelnen Herstellern. Daher sollen sie selbst entscheiden können, welche Bauteile sie auf ihr lokales System herunterladen.

Anforderung	**3.2) Konfiguration von Berechtigungen zum Zugriff auf Bauteildaten**
Beschreibung	Für bereitgestellte Bauteildaten müssen verschiedene Berechtigungen vergeben werden können.
Stakeholder	Geschäftsführung
Spezifizierung	Ein Bauteildatensystem beinhaltet verschiedene Arten von Zugriffen. Es muss dementsprechend möglich sein, verschiedene Anwenderklassen mit jeweils eigenen Zugriffsrechten zu konfigurieren. Der Download von Daten soll einen Zugang erfordern, der ggf. mit der Lizenzverwaltung verknüpft werden kann.

Anforderung	**4) Internationalisierung der Bauteildaten**
Beschreibung	Bauteildaten sollen in verschiedenen Sprachen verfügbar sein.
Stakeholder	Geschäftsführung, Produktmanagement
Spezifizierung	Die Software ist bereits in mehreren Sprachen verfügbar, allerdings fehlt bisher die Übersetzung der Bauteildaten. Eine Ausnahme besteht für zwei Hersteller, deren Bibliotheken als zusätzliche Dateien in der niederländischen bzw. der portugiesischen Sprache erstellt wurden. Der Aufwand für diesen Prozess soll erheblich reduziert werden. Es kann vorkommen, dass Bauteile nur in bestimmten Ländern existieren. Bestehende Projekte eines Anwenders sollen auch in einer neu gewählten Sprache geladen werden können. Außerdem dürfen die separaten Dateien, die bisher für zwei Hersteller erzeugt wurden und auch bereits von Anwendern benutzt werden, zu keinen Problemen führen.
Teilanforderungen	4.1) Umgang mit länderspezifischen Bauteilen 4.2) Zusammenführung der bereits übersetzten Bibliotheken

Anforderung	**4.1) Umgang mit länderspezifischen Bauteilen**
Beschreibung	Länderspezifische Bauteile sollten dem Anwender entsprechend angezeigt werden.
Stakeholder	Produktmanagement, Anwender
Spezifizierung	Manche Bauteile existieren nur in bestimmten Ländern. Es soll ein Weg gefunden werden, den Anwender über die jeweilige Verfügbarkeit verschiedener Bauteile in seinem Land mindestens zu informieren oder ihm ausschließlich verfügbare Bauteile anzuzeigen.

Anforderung	**4.2) Zusammenführung der bereits übersetzten Bibliotheken**
Beschreibung	Die bereits übersetzten und dadurch getrennten Bibliotheken sollen zusammengeführt werden.
Stakeholder	Produktmanagement
Spezifizierung	Für zwei Hersteller wurden bereits übersetzte Bibliotheken erstellt. Diese liegen als separate Dateien mit eigenen IDs für Bauteile vor und werden bereits von Anwendern benutzt. Bei einer generellen Lösung für die Internationalisierung von Bauteildaten müssen die Bibliotheken zusammengeführt werden und die Projekte der Anwender weiterhin funktionieren.

Anforderung	**5) Anpassung von Metadaten der Bauteile durch den Anwender**
Beschreibung	Die Anwender sollen für die Materialliste extrinsische Metadaten (Preis, Artikelnummer, Gewicht, Bezeichnung) bearbeiten können.
Stakeholder	Anwender
Spezifizierung	Preise, Artikelnummern, Gewichte und Bezeichnungen von Bauteilen werden beim Export der Materialliste verwendet. Bereits implementiert ist in der Materialliste die Anpassung der Bezeichnung. Anwender sollen auch die restlichen extrinsischen Metadaten anpassen können. Dies muss mit verschiedenen Sprachen harmonieren. Neue, geänderte oder gelöschte Bauteile bei Updates passen dabei möglicherweise nicht zu den vom Kunden gewünschten Einstellungen. Wenn sich Bauteile vom Hersteller ändern, sollte der Anwender weiterhin die alten Bauteile mit seinen Bezeichnungen verwenden können.

Anforderung	**6) Austausch von Bauteilen zwischen Anwendern**
Beschreibung	Anwender können benutzerdefinierte Bauteile erstellen und sollen diese untereinander tauschen können.
Stakeholder	Anwender
Spezifizierung	Über den programminternen Bauteileditor ist es für Anwender möglich, benutzerdefinierte Bauteile zu erstellen. Es soll eine Möglichkeit geschaffen werden, diese Bauteile mit anderen Anwendern teilen zu können. Der Austausch sollte jedoch bestimmten Regulierungen und Prüfungen unterliegen.

Anforderung (nicht-funktional)	**I) Erhaltung der Performance**
Beschreibung	Die Performance der Software soll grundsätzlich hoch sein.
Stakeholder	Geschäftsführung, Produktmanagement, Anwender
Spezifizierung	Für jede Änderung oder Erweiterung der Software muss gewährleistet sein, dass sie keine signifikanten Einbußen in der grundlegenden Performance des Programms zur Folge hat.

Anforderung (nicht-funktional)	**II) Erweiterbarkeit der Software**
Beschreibung	Die Software soll stets leicht erweiterbar sein.
Stakeholder	Geschäftsführung, Produktmanagement
Spezifizierung	Bei der Architektur der Software und ihren Einzelkomponenten soll eine Struktur gewählt werden, die leichte Änderungen und Erweiterungen zulässt.

3.3 Analyse

Für die vorliegenden Anforderungen muss eine individuelle Priorisierung festgelegt werden, um zu bestimmen, welche der Anforderungen aktuell tatsächlich für eine Umsetzung beachtet werden sollen. Die Anforderungen bestehen bislang unabhängig von möglichen Implementierungsansätzen. Da jedoch nicht nur der jeweilige Nutzen, sondern auch die Kosten und Risiken zu ermitteln sind, werden aus den Anforderungen zunächst konkrete Features abgeleitet, die entweder eine Änderung oder eine Erweiterung des Bauteildatensystems darstellen. Diese Features können im Hinblick auf ihren Nutzen, ihre Kosten und ihre Risiken bewertet werden, um schließlich die präzisen Anforderungen zu formulieren, welche das zu konzipierende System zwingend erfüllen muss.

3.3.1 Strukturierung

Um aus den Anforderungen konkrete Änderungen oder Erweiterungen abzuleiten, müssen sie zunächst sinnvoll gruppiert werden. Die Abbildung 3.1 gibt einen Überblick zu allen ermittelten Anforderungen. Dabei werden alle Anforderungen, die mindestens eine Teilanforderung besitzen, unterstrichen. Eine durchgezogene Linie zu einer Teilanforderung bedeutet dabei, dass diese zwingend notwendig für das Erfüllen der unterstrichenen Anforderung ist. Eine optionale Teilanforderung wird mit der Verbindung durch eine gestrichelte Linie dargestellt.

Werden die Anforderungen nach ihrem jeweiligen Scope zusammengefasst, so entstehen insgesamt vier Gruppen. Der *Scope* bezeichnet dabei den Bereich der Programmstruktur, den die Umsetzung einer Anforderung beträfe, sowie die Art der Änderung aus Sicht der Softwarearchitektur. Die einzelnen Gruppenzugehörigkeiten sind in der Abbildung farblich getrennt und werden in der folgenden Tabelle dementsprechend angeordnet. Notwendige Teilanforderungen werden als Teil der zugehörigen Anforderung betrachtet und nicht weiter explizit aufgeführt. Aus der

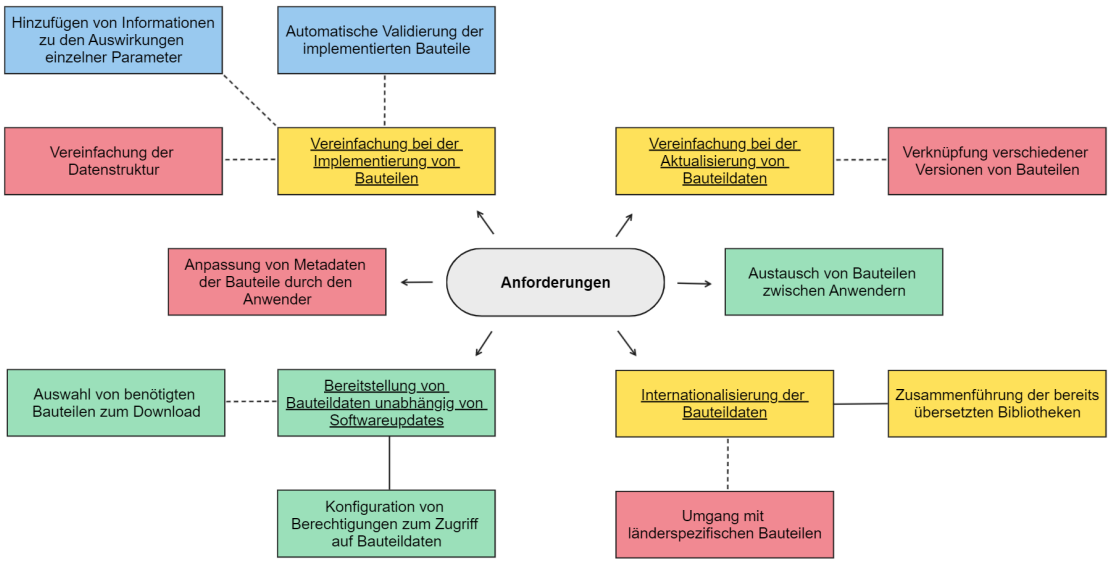


Abbildung 3.1: Anforderungen an die Bauteilbibliothek

Gruppierung der Anforderungen lassen sich jeweils potenzielle Änderungen oder Erweiterungen ableiten, welche in Kapitel 3.3.3 detailliert erläutert und schließlich bewertet werden. Die Ideen zu Änderungen und Erweiterungen wurden möglichst pragmatisch gewählt, sodass sie zur Erfüllung der jeweiligen Anforderung mit einem möglichst überschaubaren Kostenaufwand beitragen könnten.

Zur leichteren Referenzierung werden die Änderungen und Erweiterungen alphabetisch gegliedert.

Scope	**intern**
Beschreibung	Die Anforderungen erfordern eine Änderung oder Erweiterung der internen Programmstruktur.
Anforderungen	1.1) Vereinfachung der Datenstruktur 2.1) Verknüpfung verschiedener Versionen von Bauteilen 4.1) Umgang mit länderspezifischen Bauteilen 5) Anpassung von Metadaten der Bauteile durch den Anwender
Änderungen/ Erweiterungen	A) Verschachtelung der Elemente im XML-Format B) Migration in ein neues Dateiformat C) Bauteileigenschaft „Predecessor“ D) Bauteileigenschaften für benutzerdefinierte Metadaten E) Bauteileigenschaft „LocalAvailability“

Scope	**extern (strukturabhängig)**
Beschreibung	Die Anforderungen erfordern eine Erweiterung der Software, die eine direkte Anbindung an die interne Programmstruktur besitzt.
Anforderungen	1) Vereinfachung bei der Implementierung von Bauteilen 2) Vereinfachung bei der Aktualisierung von Bauteildaten 4) Internationalisierung der Bauteildaten
Änderungen/ Erweiterungen	F) Benutzeroberfläche zum Erstellen von Bauteilen G) Auslagerung von Preisen H) Auslagerung von Sprachressourcen

Scope	**extern (unabhängig)**
Beschreibung	Die Anforderungen erfordern eine Erweiterung der Software, die weitestgehend unabhängig von der internen Programmstruktur ist.
Anforderungen	3) Bereitstellung von Bauteildaten unabhängig von Softwareupdates 3.1) Auswahl von benötigten Bauteilen zum Download 6) Austausch von Bauteilen zwischen Anwendern
Änderungen/ Erweiterungen	I) Upload von Bauteilen J) Download von Bauteilen

Scope	**ohne Anwenderfunktion**
Beschreibung	Die Anforderungen stehen in keinem direkten Zusammenhang zur Funktionsweise der Software aus Anwendersicht.
Anforderungen	1.2) Hinzufügen von Informationen zu den Auswirkungen einzelner Parameter 1.3) Automatische Validierung der implementierten Bauteile
Änderungen/ Erweiterungen	K) Tooltips beim Erstellen von Bauteilen L) Validierung von Bauteilen

Die Umsetzung der Anforderungen ohne Anwenderfunktion stellt ausschließlich eine Unterstützung der Entwickler dar. Der konkrete Aufwand und Nutzen ist stark abhängig von der gewählten Umgebung, in der zukünfig neue Bauteile implementiert werden. Entsteht hierfür z. B. eine dynamische Benutzeroberfläche, lassen sich derartige Erweiterungen günstig umsetzen, während das aktuelle Vorgehen dies nur schwer zulässt. Somit lassen sich die Anforderungen ohne Anwenderfunktion nicht im Vorhinein bewerten und werden daher im weiteren Vorgehen nicht weiter beachtet. Sie können dennoch für die Auswahl eines Lösungsentwurfs von Bedeutung sein – ebenso wie die am Ende von Kapitel 3.2 aufgeführten nicht-funktionalen Anforderungen.

3.3.2 Methodik

Um die Wichtigkeit der einzelnen Änderungen/Erweiterungen zu bewerten, werden sie im Hinblick auf verschiedene Kriterien untersucht. Dabei geht es prinzipiell um eine Gegenüberstellung von Nutzen und Kosten sowie Risiken einer potenziellen Umsetzung. Diese Parameter müssen anhand diverser Einzelkriterien bewertet werden. Durch eine festgelegte Gewichtung der Kriterien ergibt sich letztendlich ein Gesamtergebnis für den Nutzen und die Kosten/Risiken, auf dessen Grundlage eine Aussage über die individuelle Priorisierung des Features bzw. der zugrundeliegenden Anforderungen getroffen werden kann.

Der Nutzen eines Features setzt sich hierbei aus folgenden Kriterien zusammen:

- interner Nutzen
  (Prozessvereinfachung, Prozessbeschleunigung, Fehlerprävention)
- externer Nutzen
- sofortiger Nutzen
- strategischer Nutzen

In den drei Fällen des internen Nutzens, die etwa gleich stark zu gewichten sind, geht es um eine Form der langfristigen Kostenreduktion. Der externe Nutzen umfasst sämtlichen Nutzen aus Anwendersicht und führt zu einer möglichen Umsatzerhöhung. Da die Kundenzufriedenheit insgesamt bereits hoch ist und die Kostenreduktion aktuell das größere Potenzial zur Gewinnoptimierung bietet, wird der interne Nutzen insgesamt höher gewichtet als der externe Nutzen.

Der angestrebte Nutzen ist bei allen Anforderungen vor allem von langfristiger Natur, weshalb der sofortige Nutzen in der Gewichtung nur eine untergeordnete Rolle einnimmt. Der strategische Nutzen wird gesondert betrachtet, da er sich sowohl intern als auch extern verorten lässt und aus Unternehmenssicht ein großes langfristiges Potenzial bietet (Beispiel: Erschließen neuer Kundenkreise).

Für die Kosten und Risiken werden folgende Kriterien definiert:

- einmalige Kosten
- Instandhaltungskosten
- kurzfristige Risiken
- langfristige Risiken

Die einmaligen Kosten werden anhand von Story Points (SP) geschätzt. Dabei handelt es sich jeweils um einen relativen Wert, um die Kostenverhältnisse zwischen den verschiedenen Features darzulegen. Die Instandhaltungskosten umfassen vor allem potenzielle zukünftige Aufwände für Anpassungen des Systems bei einer Veränderung der Software. Diese Kosten sollten bei einer leichten Änder- und Erweiterbarkeit grundsätzlich niedrig ausfallen und werden daher deutlich höher gewichtet als die einmaligen Kosten. Die Umsetzung eines Features birgt vor allem dann Risiken in sich, wenn es die Änderung von bereits vorhandenen Strukturen umfasst. Langfristige Risiken sind hierbei als besonders kritisch anzusehen und fallen dementsprechend ebenfalls höher ins Gewicht.

Um nun einen aussagekräftigen Wert für die Wichtigkeit des Features zu erhalten, wird der Nutzen einer Umsetzung schließlich gegen die Kosten und Risiken abgewogen. Die Risiken werden auf der Kostenseite aus mathematischer Sicht nicht als Summand, sondern als Faktor betrachtet, da sehr hohe Risiken selbst bei hohem Nutzen und geringen Grundkosten zu erheblichen Zusatzkosten führen können und somit ein Ausschlusskriterium darstellen.

3.3.3 Bewertung

Änderung/ Erweiterung	**A) Verschachtelung der Elemente im XML-Format**
Beschreibung	Die XML-Struktur wird so verändert, dass die Hierarchie der Objekte durch eine Verschachtelung realisiert wird. Dadurch wird die Datenmenge komprimiert und die meisten Informationen zu einem Bauteil stehen an einer Stelle.
Nutzen	schnellere Implementierung von Bauteilen, geringere Fehleranfälligkeit
Kosten	Konvertierung der alten Struktur, Anpassungen bei diversen Funktionen der Software $\Rightarrow$ 5 SP + mögliche Instandhaltungskosten
Risiken	Regressionsfehler durch strukturelle Änderungen

Änderung/ Erweiterung	**B) Migration in ein neues Dateiformat**
Beschreibung	Die XML-Struktur wird ein anderes Dateiformat (z. B. JSON) migriert. Dabei können diverse Änderungen an der Struktur vorgenommen werden.
Nutzen	höhere Praktikabilität für Objektorientierung, möglicherweise höhere Flexibilität für Änderungen und Erweiterungen
Kosten	Konvertierung der alten Struktur, Anpassungen bei diversen Funktionen der Software $\Rightarrow$ 13 SP + mögliche Instandhaltungskosten
Risiken	massive Regressionsfehler durch strukturelle Änderungen, langfristige Vor- und Nachteile ungewiss

Änderung/ Erweiterung	**C) Bauteileigenschaft „Predecessor“**
Beschreibung	Bauteile erhalten eine neue Eigenschaft „Predecessor“, in der ein Vorgängermodell des Bauteils referenziert werden kann. Die Eigenschaften werden dann an das Nachfolgemodell vererbt und können optional angepasst werden.
Nutzen	ermöglicht Umgang mit verschiedenen Versionen eines Bauteils
Kosten	Implementierung der entsprechenden Funktionen ⇒ 1 SP
Risiken	keine

Änderung/ Erweiterung	**D) Bauteileigenschaften für benutzerdefinierte Metadaten**
Beschreibung	Bauteile erhalten neue Eigenschaften für benutzerdefinierte Metadaten. Diese werden beim Export der Materialliste verwendet.
Nutzen	ermöglicht individuelle Anpassung der Metadaten
Kosten	Implementierung der entsprechenden Funktionen ⇒ 3 SP
Risiken	Fehler durch veränderte Bauteile nach Updates

Änderung/ Erweiterung	**E) Bauteileigenschaft „LocalAvailability“**
Beschreibung	Bauteile erhalten eine neue Eigenschaft „LocalAvailability“, die angibt, in welchen Sprachen ein länderspezifisches Bauteil verfügbar sein soll.
Nutzen	ermöglicht Umgang mit länderspezifischen Bauteilen, vereinfacht Internationalisierung
Kosten	Implementierung der entsprechenden Funktionen ⇒ 1 SP
Risiken	keine

Änderung/ Erweiterung	**F) Benutzeroberfläche zum Erstellen von Bauteilen**
Beschreibung	Eine Benutzeroberfläche zum Erstellen und Bearbeiten von Bauteildaten wird erstellt. Darin können vorhandene Bauteile als Vorlage genutzt und fehlerhafte Eingaben ausgeschlossen werden.
Nutzen	Vereinfachung des Prozesses, ermöglicht weitere Hilfestellungen (z. B. Tooltips) für Entwickler
Kosten	Implementierung der Benutzeroberfläche $\Rightarrow$ 21 SP + Instandhaltungskosten
Risiken	ungewiss, ob Bauteile dadurch tatsächlich schneller erstellt werden können

Änderung/ Erweiterung	**G) Auslagerung von Preisen**
Beschreibung	Die Preise der Bauteile werden ausgelagert, sodass sie besser aktualisiert werden können.
Nutzen	geringerer Aufwand zur Preisaktualisierung, mehr Möglichkeiten für direkten Import
Kosten	Erweiterung der Struktur $\Rightarrow$ 2 SP
Risiken	langfristiger Nutzen ungewiss aufgrund verschiedener Formate der Hersteller

Änderung/ Erweiterung	**H) Auslagerung von Sprachressourcen**
Beschreibung	Die Sprachressourcen der Bauteildaten werden ausgelagert, sodass außerhalb der eigentlichen Datenstruktur eine effiziente Übersetzung stattfinden kann. Die übersetzten Ressourcen werden dann je nach Sprachauswahl automatisch in die Bibliothek des Anwenders geladen.
Nutzen	geringerer Aufwand zur Internationalisierung der gesamten Bauteilbibliothek
Kosten	Erweiterung durch eine separate Struktur, Zusammenführung der bereits übersetzten Bibliotheken $\Rightarrow$ 5 SP + mögliche Instandhaltungskosten
Risiken	Fehler durch bereits übersetzte Bibliotheken, Fehler bei benutzerdefinierten Bauteilen

Änderung/ Erweiterung	**I) Upload von Bauteilen**
Beschreibung	Durch Anbindung an einen Server können Bauteile hochgeladen werden.
Nutzen	ermöglicht Austausch von Bauteilen zwischen den Anwendern
Kosten	Implementierung der Plattform und der entsprechenden Funktionen, Konfiguration von Berechtigungen $\Rightarrow$ 8 SP + hohe Instandhaltungskosten
Risiken	fehlerhafte Nutzung der Funktion, Inhalte müssen geprüft werden

Änderung/ Erweiterung	**J) Download von Bauteilen**
Beschreibung	Durch Anbindung an einen Server können Bauteile heruntergeladen werden.
Nutzen	Erhalt von Bauteilen unabhängig von Softwareupdates, ermöglicht benutzerdefinierte Auswahl
Kosten	Implementierung der Plattform und der entsprechenden Funktionen, Konfiguration von Berechtigungen $\Rightarrow$ 8 SP + Instandhaltungskosten
Risiken	keine

3.3.4 Priorisierung

In der Kosten-Nutzen-Matrix (siehe Abbildung 3.2) werden die potenziellen Änderungen/Erweiterungen als einzelne Punkte in einem Koordinatensystem dargestellt, welches den Nutzen und die Kosten/Risiken als Achsen besitzt, und anhand dessen in drei Prioritätsstufen aufgeteilt:

- hoch:
  hoher Nutzen, geringe Kosten/Risiken
- mittel:
  ausgewogenes Verhältnis zwischen Nutzen und Kosten/Risiken
- niedrig:
  geringer Nutzen, hohe Kosten/Risiken

Features mit einer hohen Priorität sollen durch das zukünftige Bauteildaten-Verwaltungssystem umgesetzt oder mindestens ermöglicht werden. Features mit einer mittleren Priorität sollen möglichst in der Form berücksichtigt werden, dass eine Grundlage geschaffen wird, die sich langfristig mit deren Umsetzung vereinbaren lässt.

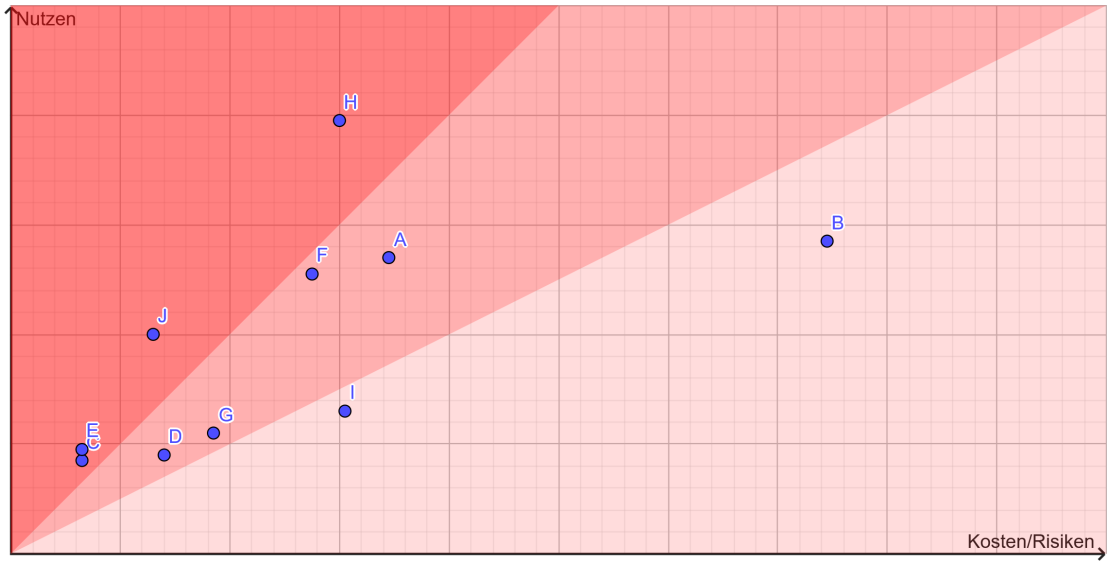


Abbildung 3.2: Kosten-Nutzen-Matrix

Aus der Matrix ergibt sich folgende Priorisierung:

Prioritätsstufe	**Änderungen/Erweiterungen**
hoch	H) Auslagerung von Sprachressourcen J) Download von Bauteilen E) Bauteileigenschaft „LocalAvailability“ C) Bauteileigenschaft „Predecessor“
mittel	F) Benutzeroberfläche zum Erstellen von Bauteilen A) Verschachtelung der Elemente im XML-Format D) Bauteileigenschaften für benutzerdefinierte Metadaten G) Auslagerung von Preisen
niedrig	I) Upload von Bauteilen B) Migration in ein neues Dateiformat

4 Datenbanksystem als Lösungsentwurf

Als Lösungsansatz zur Problemstellung wird im Folgenden ein relationales Datenbanksystem inklusive einer REST-API (REST für „*Representational State Transfer*“, API für „*Application-Programming-Interface*“) entworfen. Dieses System soll die zukünftige Grundlage zur flexiblen Verwaltung von Bauteildaten bilden. Zusätzlich wird ein Client mit Benutzeroberfläche konzipiert, der über die von der API bereitgestellten HTTP-Abfragen mit der Datenbank kommunizieren kann und bereits einige Grundfunktionalitäten beinhaltet.

Aufgrund der hohen Komplexität des Projekts beschränkt sich die Umsetzung des Systems im Rahmen dieser Arbeit auf einen Prototyp. Der konkrete Umfang dessen wird in Kapitel 4.2 definiert. Dabei wird sichergestellt, dass das Produkt funktioniert und gleichzeitig Erweiterungen zulässt, ohne dafür die fundamentale Struktur der Datenbank nachträglich verändern zu müssen. Im Nachgang werden die noch offenen Umsetzungsschritte sowie mögliche Anpassungen für die nachfolgenden Iterationen der Produktentwicklung diskutiert.

4.1 Begründung der Auswahl

Die Anforderungsanalyse ergab insgesamt eine hohe Priorisierung derjenigen Anforderungen, welche die Online-Bereitstellung oder die Internationalisierung der Bauteildaten betreffen. Beide dieser Anforderungsbereiche lassen sich durch ein Datenbanksystem mit angebundener REST-API nahezu vollständig abdecken. Bei einer entsprechenden Architektur mit Trennung der Funktionalitäten lassen sich zudem neue Felder wie Bauteileigenschaften ohne erheblichen Aufwand hinzufügen, womit ebenfalls eine Reihe von wichtigen Anforderungen erfüllt werden kann.

In einer relationalen Datenbank kann die Struktur der Bauteildaten, welche konzeptuell ohnehin bereits einer Datenbankstruktur ähnelt, 1:1 abgebildet werden. Zusätzliche Tabellen mit Sprachressourcen können die Basis für die Internationalisierung der Daten bilden. Außerdem kann eine Datenbank zentral verwaltet werden, ermöglicht Zugriff durch verschiedene Benutzergruppen und bietet grundsätzlich eine hohe Performance bei den nötigen Abfragen.

Durch die Anbindung einer REST-API lassen sich entsprechende HTTP-Abfragen derart konfigurieren, dass die Daten in einer geeigneten Struktur zwischen Datenbank und Client transferiert werden können. So ließen sich beispielsweise die Daten einer bestimmten Herstellerbibliothek in einer benutzerdefinierten Sprache in das XML-Format überführen.

4.2 Umfang

Für die Entwicklung eines Prototyps geht es darum, die Basis für das spätere System zu schaffen, indem die grundlegenden Strukturen festgelegt werden. Die Implementierung einiger Grundfunktionalitäten sorgt dafür, dass eine möglichst konkrete Vorstellung von der Anwendung und vom zukünftigen Mehrwert des Systems entstehen kann.

Der Fokus liegt hierbei auf der Erfüllung der folgenden Aspekte:

- Die Datenbank soll so aufgebaut werden, dass alle Elemente eindeutig identifizierbar sind.
- Die Datenbank soll alle bisherigen Bauteildaten von sämtlichen Herstellern derart abspeichern können, dass ein verlustfreier Austausch zwischen der Datenbankstruktur und der Bibliothekstruktur in CP-Pro CAD stattfinden kann.
- Die Datenbank soll Informationen zu Bauteilen in verschiedenen Sprachen abspeichern können, ohne dass komplett neue Bauteile hierfür angelegt werden müssen.
- Die REST-API soll alle nötigen HTTP-Abfragen bereitstellen, damit theoretisch alle Daten aus der Datenbank gelesen werden können.
- Die REST-API soll alle nötigen HTTP-Abfragen bereitstellen, damit neue Elemente beliebig zur Datenbank hinzugefügt werden können.
- Die REST-API soll den Umgang mit Sprachressourcen ermöglichen.
- Der Client soll Bauteildaten aus der Datenbank im XML-Format herunterladen können, sodass sie mit der Software CP-Pro CAD kompatibel sind.
- Der Client soll eine Möglichkeit bieten, um neue Übersetzungen zu Bauteildaten in die Datenbank einzutragen.

4.3 Umsetzung

Das System zur Bauteildaten-Verwaltung besteht aus drei grundlegenden Komponenten: Datenbank, REST-API und Client. Die Datenbank ist auf einem Server abgelegt und speichert dort sämtliche Bauteildaten. Die REST-API dient als Schnittstelle und kann die Daten aus der Datenbank strukturiert an den Client ausliefern, der lokal beim Anwender installiert wird und dort eine Anbindung zur bestehenden Software CP-Pro CAD besitzt. Nachfolgend wird der Aufbau der einzelnen Komponenten genauer erklärt.

4.3.1 Datenbank

Als Basis des Systems wurde eine Datenbank gewählt, welche auf einer Architektur von *Microsoft SQL Server 2022* aufgebaut ist. Dabei handelt es sich um ein relationales Datenbankmanagementsystem unter der Einhaltung aktueller SQL-Standards.

Die Datenbank wurde zunächst so konzipiert, dass sie die in Kapitel 2.2.2 erläuterte Datenstruktur aus der Bauteilbibliothek von CP-Pro CAD abbildet. Dieses Schema wurde um einige Tabellen, Attribute und Beziehungen ergänzt, welche zum einen die Eindeutigkeit der Elemente gewährleisten und zum anderen der Auslagerung der Sprachressourcen dienen. Im Ausschnitt des Datenbankdiagramms sind die Ergänzungen zur ursprünglichen Datenstruktur exemplarisch zu erkennen (siehe Abbildung 4.1).

Da in der ursprünglichen XML-Struktur, welche programmintern weiterhin verwendet wird, grundsätzlich eine Datei pro Hersteller existierte, waren die IDs der einzelnen Elemente zwar lokal eindeutig (innerhalb einer Datei), jedoch nicht global. Eine hierzu äquivalente Struktur würde das Anlegen separater Datenbanken für jeden Hersteller erfordern. Um diese ineffiziente Architektur zu vermeiden, wurde

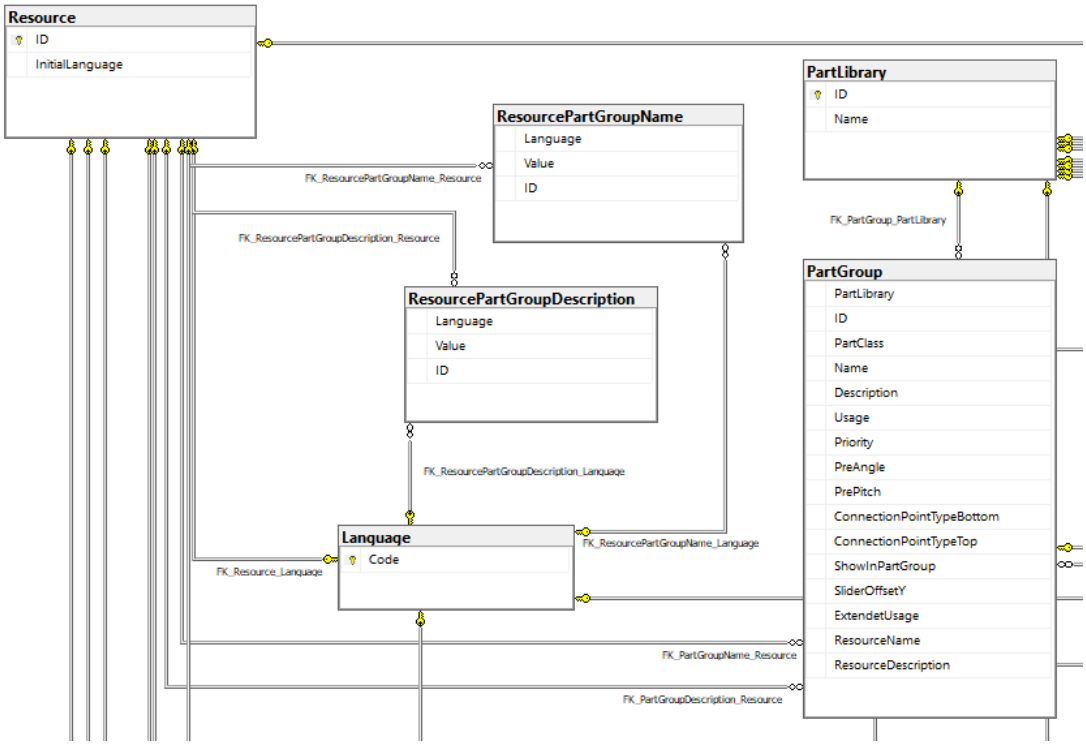


Abbildung 4.1: Ausschnitt des Datenbankdiagramms

jede Tabelle um ein Attribut ergänzt (*PartLibrary*), welches die ID der Bibliothek des jeweiligen Herstellers enthält. In einer neuen Tabelle wird jeder solchen ID ein Name zugewiesen, welcher dem Namen der entsprechenden XML-Datei entspricht. Die ursprünglichen IDs der einzelnen Elemente bleiben bestehen und bilden nun gemeinsam mit den neuen PartLibrary-IDs eindeutige Schlüssel.

Um die Internationalisierung der Bauteilbibliothek zu ermöglichen, werden die Sprachressourcen in neue Tabellen ausgelagert. Diese enthalten jeweils drei Spalten: *ID*, *Language* und *Value*. Die ID kennzeichnet die Ressource, die Language wird mit einem einheitlichen Code angegeben und der Value enthält letztendlich die Übersetzung in der passenden Sprache. Um die Beziehungen zu den Objekten herzustellen, dienen die IDs als Fremdschlüsselattribute in den entsprechenden Tabellen (Beispiel: *ResourceDescription* in *PartGroup* als Referenz zur Ressource der Beschreibung einer Bauteilgruppe). Zur möglichst einfachen Sprachverwaltung

wurde außerdem eine Tabelle für die Language-Codes erstellt, und in einer weiteren Tabelle wird für jede neu angelegte Ressource die Originalsprache abgespeichert (*InitialLanguage*).

4.3.2 REST-API

Für die REST-API wurde die plattformübergreifende Laufzeitumgebung *Node.js* verwendet, welche es erlaubt, JavaScript-Code auf einem Webserver auszuführen. Hierbei kam das Modul *Express* zum Einsatz, welches ein Framework enthält, das sich für Webanwendungen eignet. Es stellt einen Router zur Verfügung, um HTTP-Abfragen für gewählte URLs zu konfigurieren.

Abhängig von den in der URL gewählten Parametern stellt die API spezifische SQL-Abfragen an die Datenbank. Durch die dortige Auslagerung der Sprachressourcen in eigene Tabellen kann auf die Übersetzungen in der jeweiligen Sprache zugegriffen werden. Beim Download werden die Daten durch die API zusammengesetzt und dem Client mit Hilfe eines Parsers in der erforderten Struktur ausgeliefert. Beim Anlegen neuer Datensätze, welches bestimmte Berechtigungen erfordert, validiert die API das korrekte Format der Daten und fügt die neuen Sprachressourcen in die separaten Tabellen ein.

Die automatisierte Dokumentation der REST-API, welche mit Hilfe des Werkzeugs *Swagger* erzeugt wurde, gibt einen Überblick zu den konfigurierten Routen (siehe Abbildung 4.2). Darin enthalten sind verschiedene GET-Abfragen, um Daten wahlweise im XML-Format oder im JSON-Format abzufragen, sowie mehrere POST-Abfragen zum Anlegen neuer Elemente und Sprachen in der Bibliothek. Hinzu kommt eine PATCH-Abfrage, welche dazu dient, fehlende Übersetzungen in vorhandene Sprachressourcen einzufügen oder diese zu ändern.

Swagger

Part Library API 1.0.0 OAS 2.0

[Base URL: localhost:8081/]

Schemes

HTTP

default

GET	/api-docs
GET	/json/lib
GET	/json/{lib}/{lang}/{type}
POST	/json/{lib}/{lang}/{type}
GET	/json/{lib}/{lang}/{type}/{id}
GET	/json/{lib}/{lang}/{parent}/{id}/{type}
GET	/json/{lib}/{lang}/{parent}/{id}/{type}/{attr}
GET	/res/json
GET	/res/json/{tbl}/{lang}
GET	/json/{type}
GET	/json/{lib}/{type}
GET	/xml/{lib}/{lang}
POST	/xml/{lib}/{lang}
GET	/xml/{lib}/{lang}/{type}
POST	/xml/{lib}/{lang}/{type}
GET	/xml/{lib}/{lang}/{type}/{id}
POST	/lib
POST	/res
PATCH	/res/{tbl}/{id}

Abbildung 4.2: Dokumentation der REST-API in Swagger

4.3.3 Client

Der Client ist eine in C# programmierte WPF-Anwendung und orientiert sich damit an der Software CP-Pro CAD. Eine Anbindung an diese ist theoretisch möglich, allerdings funktioniert das Programm auch eigenständig.

Im Klassendiagramm sind die einzelnen Klassen mitsamt ihren Feldern, Eigenschaften und Methoden zu sehen (siehe Abbildung 4.3). Die Klasse *MainWindow* erzeugt das Hauptfenster des Programms und enthält zwei auswählbare Reiter. Während im *PartLibraryManager* die Bauteilbibliotheken der verschiedenen Hersteller verwaltet und heruntergeladen werden können, ermöglicht der *ResourceManager* die Bearbeitung aller Sprachressourcen. Der *Controller* kommuniziert über HTTP-Abfragen direkt mit der REST-API und enthält sämtliche Basisfunktionalitäten, auf die von den anderen Komponenten des Programms zurückgegriffen wird. Außerdem stellt er mehrere Objektklassen zur Verfügung, die für einen zielgerichteten Umgang mit den Daten erforderlich sind.

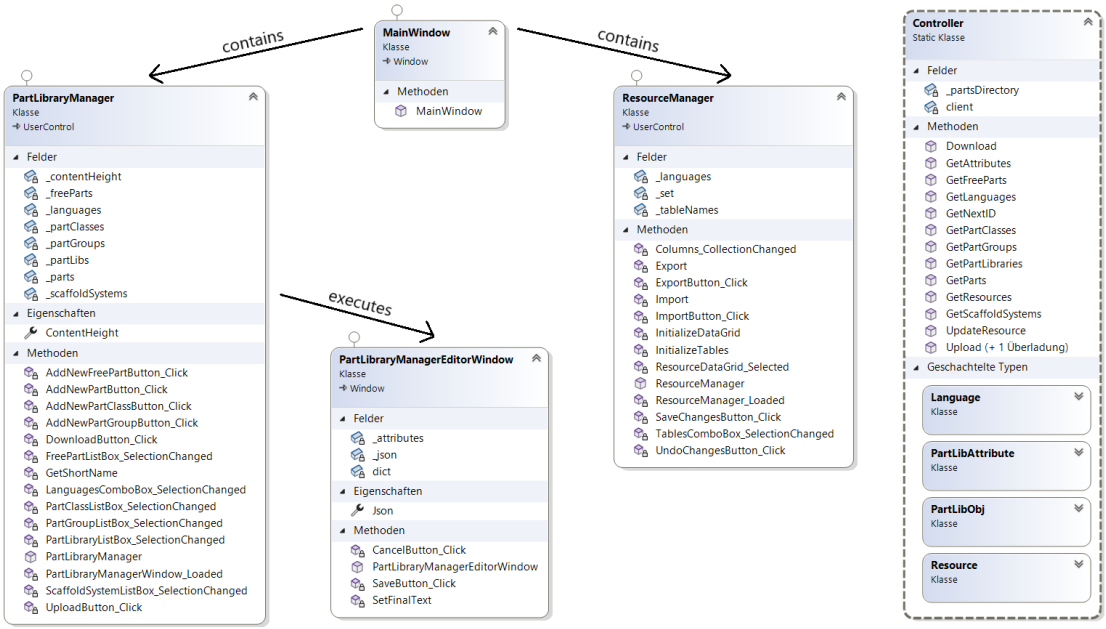


Abbildung 4.3: Klassendiagramm des Clients

Beim Start des Clients ruft die API aus der Datenbank alle Bauteilbibliotheken in der eingestellten Sprache ab und stellt diese in einem geordneten Schema dar, welches die Hierarchie der Elemente innerhalb der Bauteilbibliothek von links nach rechts absteigend widerspiegelt (siehe Abbildung 4.4). Ähnlich wie im Menü von CP-Pro CAD kann der Benutzer sich durch dieses Schema navigieren. Dadurch bekommt er einen Überblick zu allen Herstellern, Gerüstsystemen, Bauteilklassen, Bauteilgruppen, freien Bauteilen und Bauteilen, welche jeweils in Listen abgespeichert werden.

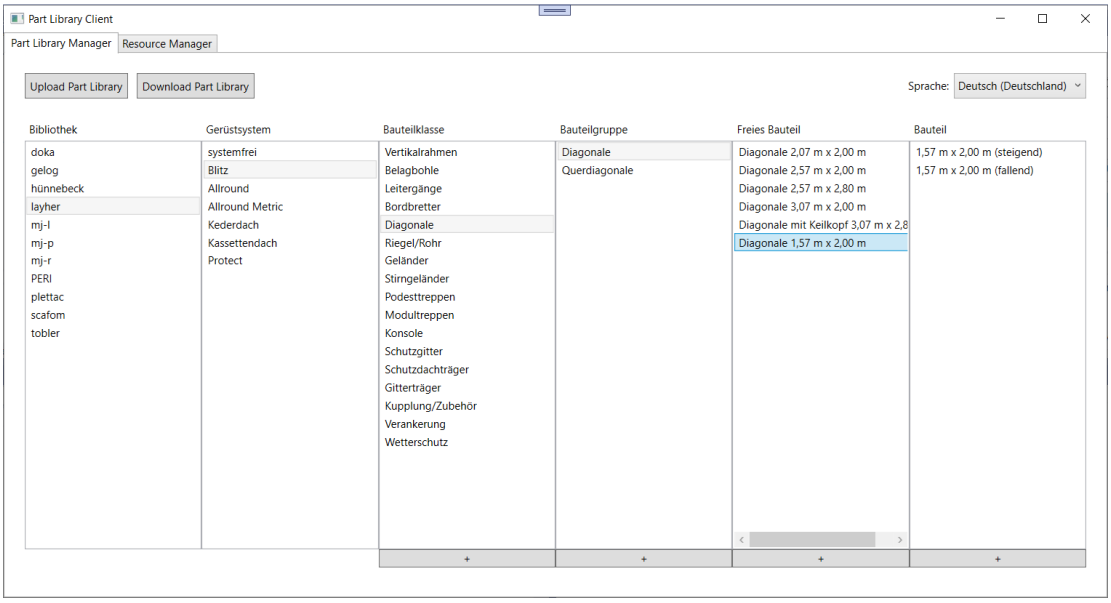


Abbildung 4.4: Übersicht auf die Bauteildaten im Client

Zu diesen Kategorien wurden Funktionen implementiert, um ein entsprechendes Element hinzuzufügen, wobei die Klasse *PartLibraryManagerEditorWindow* einen jeweils passenden Editor hierfür generiert, indem sie von der REST-API eine Liste aller Attribute erhält (siehe Abbildung 4.5). Zudem wird mit Hilfe entsprechender Abfragen automatisch eine neue ID sowie ggf. eine UUID generiert. Die IDs der PartLibrary und der hierarchisch übergeordneten Klasse des zu erstellenden Elements werden ermittelt und ebenfalls bereits im Editor eingetragen, sodass das Element korrekt zugeordnet wird.

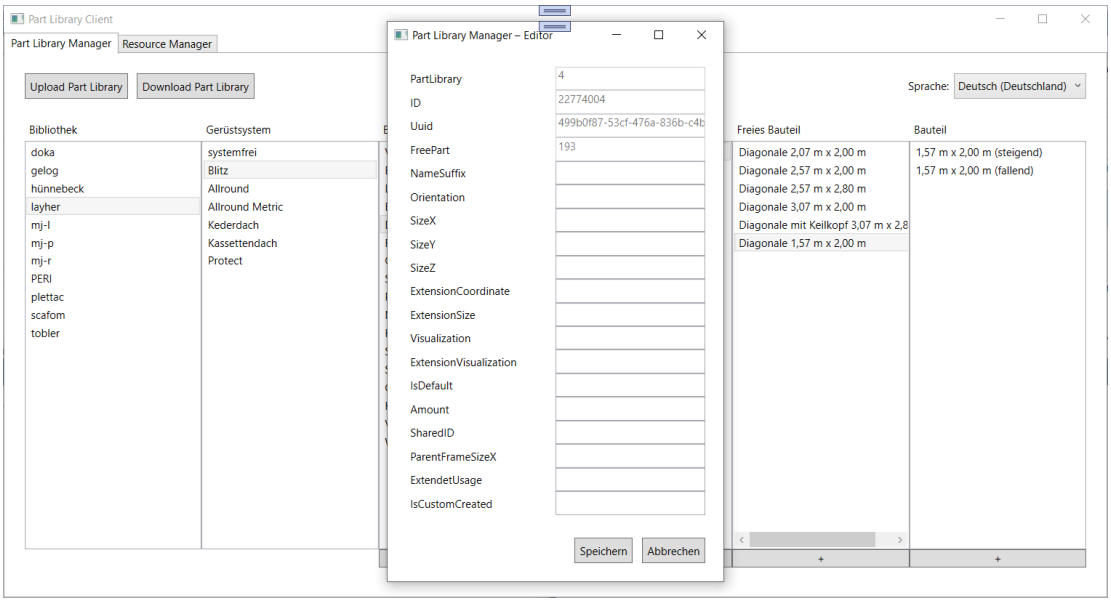


Abbildung 4.5: Hinzufügen eines neuen Bauteils in der Bibliothekenverwaltung

Die Funktionen zum Upload und Download von Bauteilbibliotheken wurden so programmiert, dass sie sowohl das JSON-Format als auch das XML-Format verarbeiten können. Dadurch ist ein flexibler Austausch von Daten zwischen Server und Client möglich. Der Dateipfad für den Download kann so konfiguriert werden, dass heruntergeladene XML-Bibliotheken eines ausgewählten Herstellers in einer ausgewählten Sprache direkt in das Programmverzeichnis von CP-Pro CAD eingefügt werden. Sie werden von der Software automatisch erkannt und sind sofort kompatibel, da sie exakt der dort vorhandenen Struktur entsprechen.

In der Ressourcenverwaltung werden alle Sprachressourcen von der API abgefragt und in einen Datensatz geladen, sodass sie tabellarisch dargestellt werden können (siehe Abbildung 4.6). Dem Anwender wird zum einen die Möglichkeit geboten, fehlende Übersetzungen direkt per Hand in die Tabelle einzutragen. Änderungen können hierbei jederzeit gespeichert oder verworfen werden. Zum anderen wurden Funktionen zum Import und Export von CSV-Dateien implementiert, sodass Übersetzungen auch extern vorgenommen werden können und somit eine hohe Kompatibilität zu anderen Systemen besteht.

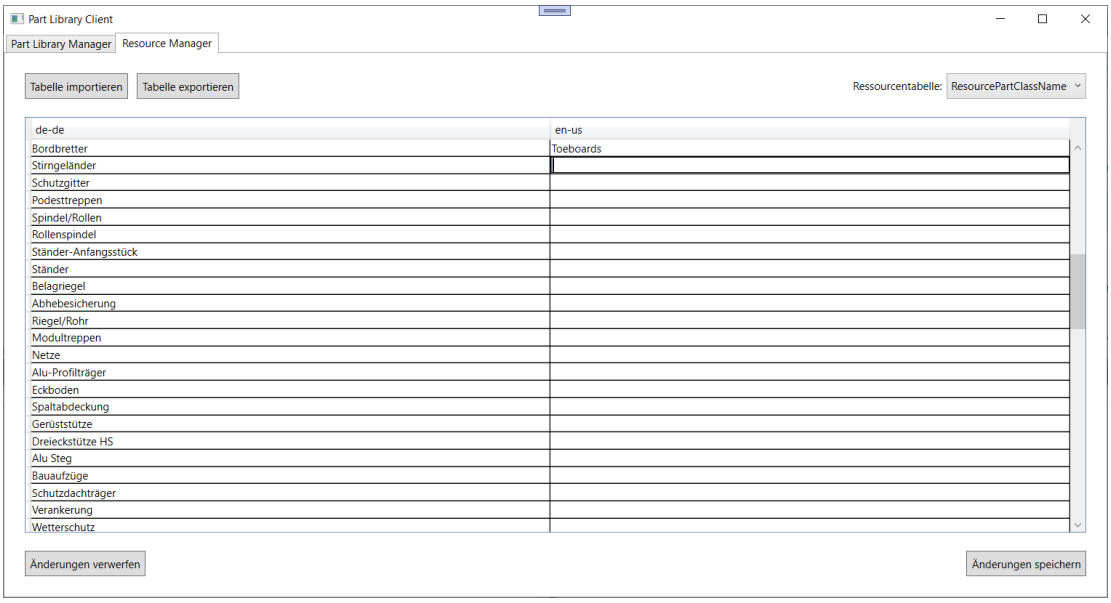


Abbildung 4.6: Eintragen von Übersetzungen in der Ressourcenverwaltung

Der Client wurde in seiner Architektur so konzipiert, dass die Funktionalitäten der verschiedenen Bereiche möglichst klar voneinander getrennt wurden. Das Layout der einzelnen Fenster/Reiter wurde in den XAML-Dateien definiert und somit vom restlichen Code separiert. Der Controller ist die einzige Instanz, die direkt mit der REST-API kommuniziert, wodurch sich die anderen Komponenten leicht verändern lassen.

4.4 Diskussion und Ausblick

Das entwickelte System wurde während seiner bisherigen Entwicklung stetig getestet, wobei auch die originalen Bauteildaten aus CP-Pro CAD Verwendung fanden. Die Tests wurden komponentenweise durchgeführt, wodurch sichergestellt wurde, dass die REST-API auch für diverse Features geeignet ist, die bisher noch nicht im Client implementiert wurden, aber potenziell in der Zukunft Erweiterungen darstellen könnten. In seiner jetzigen Form beherrscht der Prototyp des Systems die im Umfang beschriebenen Funktionalitäten und konnte zuletzt alle Tests fehlerfrei durchlaufen.

Die Auslagerung der Sprachressourcen in separate Tabellen der Datenbank reduziert in Verbindung mit den entsprechenden Fähigkeiten der API den Aufwand zum Übersetzen von Bibliotheken erheblich und stellt somit einen enormen Schritt zur Internationalisierung der Bauteildaten dar. Durch die Bereitstellung der Datenbank auf einem Server ist es nun möglich, die Bauteildaten unabhängig von der Software zu verwalten und sie den Anwendern zum Download zur Verfügung zu stellen. Somit erfüllt das System die beiden am höchsten priorisierten Anforderungen. Der Client beinhaltet zudem bereits erste Ansätze einer Benutzeroberfläche zum Anlegen neuer Bauteildaten, wodurch die Flexibilität der Datenverwaltung noch weiter erhöht werden kann.

Das gesamte System wurde so konzipiert, dass es unabhängig von den Attributen einzelner Objekte der Bauteilbibliothek funktioniert. Das bedeutet, dass auf Datenbankebene eine Tabelle (z. B. ScaffoldSystem) beliebig viele Spalten von beliebiger Art besitzen darf, ohne dass Funktionalitäten der REST-API oder des Clients dafür angepasst werden müssten. In dieser Hinsicht lässt sich von einer hohen Flexibilität des Systems sprechen, welche für die Zukunft einen problemlosen Umgang mit neuen Bauteileigenschaften gewährleistet. Damit können weitere wichtige Anforderungen abgedeckt werden. Außerdem sorgt diese Flexibilität für eine leichte Änderbarkeit und Erweiterbarkeit des Systems, wenn es darum geht, mit potenziellen neuen Strukturen umzugehen. So konnte bereits testweise die Generie-

rung einer XML-Datei mit Verschachtelung der Elemente mit Hilfe entsprechender Methoden der API erfolgreich durchgeführt werden.

Bevor das System für Anwender veröffentlicht wird, muss es zwingend durch ein Berechtigungssystem ergänzt werden. Sowohl auf Datenbank- als auch auf Clientseite müssen mehrere Benutzerrollen mit verschiedenen Zugriffsrechten definiert werden. Insbesondere das Anlegen neuer Bauteildaten und Übersetzungen sollte spezielle Berechtigungen erfordern. In diesem Zusammenhang wäre auch möglich, den Client in zwei (oder sogar mehr) verschiedene Programme zu trennen, da die verschiedenen Funktionsbereiche in der Regel auch von verschiedenen Benutzergruppen verwendet werden.

Bisher ist es im Client noch nicht möglich, neue Visualisierungen oder Verbindungspunkte von Bauteilen anzulegen. Hier wäre das Herstellen einer Verknüpfung zum in CP-Pro CAD enthaltenen Bauteileditor denkbar. Generell muss abgewogen werden, inwieweit eine direkte Integration des Verwaltungssystems in die bisherige Software oder zumindest eine direkte Kommunikation selbiger mit der Datenbank sinnvoll wäre. Dies würde neue Möglichkeiten zur Automatisierung von Prozessen schaffen und könnte die Arbeit mit den Daten erleichtern, da das System auf das interne Datenmodell der Software zurückgreifen könnte. Ein Nachteil läge jedoch unter anderem darin, dass die Verwaltung der Bauteildaten dann nicht unabhängig möglich wäre, was wiederum einem Verlust an Flexibilität gleichkäme.

Weitere mögliche Features, um die der Client ergänzt werden könnte, bestehen im Download ausgewählter Bauteile (anstelle einer gesamten Herstellerbibliothek) und in verschiedenen Hilfestellungen beim Anlegen von Bauteilen. Je weiter die Entwicklung einer intuitiven Benutzeroberfläche voranschreitet, desto mehr Möglichkeiten bestehen für derartige Unterstützungen. Beispielsweise könnte bereits während des Prozesses eine Validierung von neu hinzugefügten Bauteildaten erfolgen oder ein Tooltip mit Informationen zur Verwendung verschiedener Bauteilklassen eingebledet werden, womit sich weitere Anforderungen abdecken ließen.

5 Fazit

Im Zusammenhang mit der Bauteilbibliothek von CP-Pro CAD konnte eine Reihe von Anforderungen ermittelt werden, welche die einzelnen Stakeholder an das Produkt stellen. Diese Anforderungen betreffen verschiedene Bereiche der Software und erfordern jeweils individuelle Änderungen oder Erweiterungen, die zum Teil abhängig von der internen Programmstruktur sind. Die Ziele liegen insgesamt vor allem in der Vereinfachung von Prozessen beim Umgang mit Bauteildaten, der Verbesserung der Möglichkeiten für Anwender und der Gewinnung neuer Kundenkreise durch die Internationalisierung der Bauteildaten.

Nach einer gründlichen Analyse der Anforderungen im Hinblick auf den Nutzen sowie die Kosten und Risiken einer potenziellen Umsetzung konnten diese in drei verschiedene Prioritätsstufen aufgegliedert werden. Die Internationalisierung der Bauteildaten, für die eine Auslagerung der Sprachressourcen als Basis dienen soll, wurde aufgrund ihres enormen strategischen Nutzens am höchsten priorisiert. Darauf folgte die Bereitstellung von Bauteildaten zum Download, da sie einen hohen Nutzen für Anwender bietet und kaum Risiken mit sich bringt. Eine hohe Priorität wurde außerdem der Implementierung neuer Bauteileigenschaften zugewiesen, welche den Umgang mit länderspezifischen Bauteilen und Vorgängermodellen erheblich erleichtern sollen.

Als Lösung für ein flexibles Verwaltungssystem von Bauteildaten wurde schließlich ein Datenbanksystem mit angebundener REST-API entworfen. Der dokumentierte Prototyp des Systems beinhaltet bereits essenzielle Funktionalitäten, um die Internationalisierung der Bauteildaten sowie deren Online-Bereitstellung zu ermöglichen.

Für den Client wurde eine prototypische Benutzeroberfläche aufgebaut, in der Bauteildaten unabhängig von der internen Datenstruktur angelegt werden können. Zudem wurde das Produkt so konzipiert, dass es zukünftige Anpassungen und Erweiterungen zulässt, um weitere relevante Anforderungsbereiche abzudecken. Somit konnte die Grundlage für ein Verwaltungssystem geschaffen werden, welches Flexibilität in Bezug auf die Daten, aber auch im Sinne einer leichten Änderbarkeit gewährleistet.

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Bachelorarbeit mit dem Thema

Flexible Verwaltung von Bauteildaten in CP-Pro CAD
Anforderungsanalyse mit anschließendem Entwurf eines Datenbanksystems

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, wobei ich alle wörtlichen und sinngemäßen Zitate als solche gekennzeichnet habe. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Korschenbroich, den 2. Oktober 2023

M. Rolfes

Matthias Rolfes